

Beyond Namespace–Method Configuration: Evidence-Preserving Privacy API Resolution for ArkTS

ANONYMOUS AUTHOR(S)

Static taint analysis for OpenHarmony/ArkTS depends on identifying privacy-sensitive sources before propagation begins. A strict namespace–method source loader does not preserve enough evidence for property reads, package migration between `@ohos.*` and `@kit.*`, or calls whose sensitive identity is exposed through a manager/helper receiver. A missed identity suppresses the initial taint fact regardless of the downstream solver.

We present ARKPRISM, a static analysis framework that combines evidence-carrying API identity resolution, ArkTS-specific IR recovery, bounded asynchronous path recovery, and provenance-annotated audit subgraphs. Identity resolution records namespace-constrained package migration, receiver origin or type, access kind, and IR-lowered aliases. The IFDS analysis recovers initializers, lexical captures, exceptional flow, and access paths; a separate post-IFDS phase handles selected callback and Promise/async-await gaps and retains their provenance rather than presenting them as native IFDS edges. Lifecycle entries are ordered and explicitly bounded to avoid manufacturing flows through unrestricted event cycles.

A source-first audit samples 60 real projects by scale and configured-import presence, reviews every candidate occurrence before consulting detector output, and establishes 90 gold occurrences in 16 projects plus 44 confirmed zero projects. ARKPRISM recovers all 90 at their exact source locations with no additional reports (100% observed precision and recall; 95% Wilson lower bound 95.91%). On a 64-case identity benchmark, the complete resolver is the only evaluated configuration to classify all positive and matched-negative patterns. Across all 67 cases of the author-released HapBench suite, ARKPRISM attains 100.00% precision, 94.34% recall, and 97.09% F1; its strongest comparative advantage over HapFlow is specificity (100.00% versus 71.43%). A 48-case asynchronous benchmark further isolates nine Promise-then recoveries ($p = 0.00390625$). Finally, a single-build study of 1,014 projects characterizes detector evidence, typed may-paths, robustness, and analysis cost.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → *Software defect analysis*.

Additional Key Words and Phrases: OpenHarmony, ArkTS, static analysis, privacy APIs, taint analysis, call-chain analysis

ACM Reference Format:

Anonymous Author(s). 2027. Beyond Namespace–Method Configuration: Evidence-Preserving Privacy API Resolution for ArkTS. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 34 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

OpenHarmony applications use ArkTS, a TypeScript-derived language with declarative ArkUI components, framework-managed lifecycle callbacks, and system APIs exposed through both historical `@ohos.*` and newer `@kit.*` packages [28, 32, 41]. Privacy auditing in this ecosystem requires more than searching for method names. The same member name may denote application code or a platform API; a platform API may be reached through an imported namespace, a manager object returned by a factory, an IR temporary, or a field read rather than a call. Sensitive values may subsequently cross callbacks, Promise handlers, lifecycle entries, logs, storage, or network operations.

ArkAnalyzer provides reusable IR, control-flow, call-graph, and pointer-analysis facilities for ArkTS [14]. HapFlow builds an IFDS taint analysis on this foundation [15]. These systems make ArkTS analysis possible, but they leave a distinct front-end problem: a taint solver can propagate only sources that have first been identified. HapFlow’s released JSON loader resolves configured

namespace and method entries to SDK method signatures. That loader has no representation for a privacy-sensitive property read and does not by itself preserve the package/factory evidence of a call on a manager/helper receiver. Missing that evidence can suppress the initial source fact regardless of the downstream solver.

Consider `helper.createAsset(...)` after `helper` is returned by `photoAccessHelper.getPhotoAccessHelper(ctx)`. The member `createAsset` alone is insufficient: it is common application vocabulary, while the receiver origin establishes its platform meaning. Conversely, accepting every same-named member in a file importing a media package would create false positives. The analysis must therefore retain the evidence used for a match: namespace-constrained package migration, import alias, receiver origin or type, member, and access kind. A similar problem arises after IR lowering of compound APIs such as `request.agent.create`, and for fields such as `deviceInfo.brand`.

We present ARKPRISM, a static analysis system that resolves privacy API identities and assembles provenance-annotated evidence subgraphs for ArkTS. For each candidate usage, ARKPRISM computes a semantic identity over package/namespace migration, receiver evidence, member, and access kind. Indirect calls require an exact SDK type, a compatible call-target signature, a bounded definition/factory-origin chain, or an exact namespace receiver; generic members such as `on`, `start`, and `close` are never accepted from lexical namespace evidence alone. In parallel, the taint query admits only configured sources with an explicit typed carrier and runs IFDS with ArkTS IR recovery; permission-bearing operations without a sensitive output remain detector usages, not taint seeds. A bounded post-IFDS phase supplements selected callback, Promise-handler, and `async/await` paths; field-stored handlers and ArkUI builder options instead add provenance-labeled report call-graph context. The report keeps detector-local API/sink evidence, privacy-data paths, and framework-input paths distinct, linking detector occurrences only to compatible privacy-data source endpoints.

Our evaluation separates accuracy from prevalence. We first freeze a source-first sample of 60 real projects across three scale strata and two configured-import strata, enumerate executable candidates without reading ARKPRISM reports, and review every candidate at exact source location. The resulting benchmark contains 90 gold occurrences in 16 projects and 44 confirmed zero projects; ARKPRISM recovers all 90 and produces no additional reports. A 64-case positive/negative identity benchmark then isolates the contribution of import, receiver, factory, property, compound-namespace, and package-migration evidence. End-to-end accuracy is measured on all 67 cases in the author-released HapBench suite using its executable artifact as the directly comparable baseline: ARKPRISM attains 100.00% precision, 94.34% recall, and 97.09% F1, with 100% specificity. A separately specified 48-case asynchronous benchmark attributes nine Promise-then recoveries to post-IFDS supplementation. A single-build 1,014-project study finally measures detector-recognized operations, typed paths, provenance, robustness, and cost without treating corpus output as ground truth.

The evaluation protocol follows lessons from Android analysis research. ReproDroid shows that tools must be compared under the same query and benchmark subset [42]; TaintBench stores expected and unexpected flows rather than treating every undiscovered report as a false positive [37]; and mutation studies show that optimizations can introduce undocumented unsoundness [10]. Accordingly, we publish all HapBench labels and errors, distinguish incomplete runs from zero-flow results, and report precision, recall, specificity, balanced accuracy, F1, MCC, and per-category behavior. Android analyzers such as FlowDroid and PacDroid inform our design, but are not presented as numerical baselines because they analyze Android bytecode and Android framework models rather than ArkTS [5, 16].

This paper makes the following contributions:

- **Evidence-carrying API identity resolution.** We formulate privacy API recognition as a package-, receiver-, member-, and access-aware matching problem, covering direct and assigned calls, manager/helper receivers, IR aliases, and executable property reads while recording the acceptance witness of each match.
- **Carrier-aware ArkTS propagation.** We define typed source carriers and seed facts for privacy-data returns and callbacks, distinguish framework-provided inputs, and reject detector-only operations as taint seeds. ArkTS IR recovery operates inside IFDS, while bounded callback/Promise rules supplement unresolved asynchronous paths after the solver.
- **Provenance-annotated audit subgraphs.** We retain API identity, call context, detector-local sinks, configured-query paths, permissions, and source locations in one artifact while preserving which subsystem produced each edge and whether a configured source was linked to a detector usage.
- **A reproducible multi-level evaluation.** We contribute a source-first real-project occurrence benchmark, a matched 64-case identity benchmark, a complete reproduction on the official 67-case HapBench suite, a balanced 48-case asynchronous benchmark, and a clone-aware study of 1,014 completed projects. Each layer has a distinct oracle and claim boundary, while machine-readable manifests connect reported values to the evaluated build, rules, SDK, and project inventory.

2 Background and Motivation

2.1 OpenHarmony and ArkTS

OpenHarmony applications are organized around abilities, pages, ArkUI components, lifecycle methods, and event callbacks [27, 28, 41]. ArkTS preserves much of the TypeScript surface syntax, but introduces stricter typing and a declarative UI model. In ArkUI, a component’s build method describes UI structure, while lifecycle callbacks such as `aboutToAppear` and event handlers such as `onClick` are implicitly invoked by the framework. These properties make ArkTS attractive for application development, but complicate static analysis because important control-flow edges are not always explicit in source code.

Unlike traditional Android applications, where many lifecycle and callback conventions have been studied for years, OpenHarmony combines a relatively new application model with a rapidly evolving SDK. An ArkTS page may contain ordinary methods, lifecycle methods, anonymous functions, class-field callbacks, Promise handlers, and declarative UI builder code in the same file. The compiler and analyzer may lower these constructs into generated methods such as `%AM-style` anonymous methods, `%df1t`, `%statInit`, and `%instInit`. These generated methods are not noise for privacy analysis: they often contain callback assignments, page initialization, or API wrapper setup. An analysis that filters them out too aggressively can lose real paths.

OpenHarmony also exposes a broad API ecosystem for device identity, account information, clipboard access, sensors, camera, media, location, network state, Wi-Fi, Bluetooth, telephony, calendar, and other capabilities. Some APIs have migrated from historical `@ohos.*` packages to newer `@kit.*` packages, while many apps still contain mixed imports, aliases, default imports, and manager-style APIs. For example, a project may import device information from `@ohos.deviceInfo`, from `@kit.BasicServicesKit`, or through a default alias. Similarly, APIs such as `pasteboard` and `request-agent` operations may be invoked directly from a namespace or indirectly through an object returned by a manager factory.

This evolution creates a tension: exact package matching under-reports the same conceptual API under different names, while method-name-only matching over-reports common verbs. ARKPRISM

reports both namespace-method metrics (migration-compatible) and strict package-signature metrics (alias-sensitive).

2.2 Foundational Static Analysis for ArkTS

ArkAnalyzer provides ArkTS-aware code representation, IR transformation, Scene construction, call-graph construction, and basic data-flow support [14]. HapFlow builds on ArkAnalyzer to provide inter-procedural taint analysis for OpenHarmony, addressing lifecycle/callback entry modeling, closure-captured variables, and source/sink extraction [15].

ArkAnalyzer and HapFlow play different roles: ArkAnalyzer is a representation and graph-construction layer (Ark files, classes, methods, CFG, imports, call graphs); HapFlow is a taint-analysis layer (whether configured sources reach configured sinks). ARKPRISM is built between and around these layers, asking a different question: can we recover the evidence that an auditor needs to understand a privacy-sensitive behavior?

Privacy auditing needs a *behavior-level* explanation: what API is used, how it is reached, what exposure points are downstream, what category and permission are involved, and how patterns appear across a corpus. This distinction affects evaluation: a taint benchmark tests crafted source-to-sink cases, while a corpus audit needs API recognition against source evidence, call-chain coverage, sink distribution, and manual confirmation.

2.3 Information-flow Subgraphs

For each recognized privacy-sensitive API usage u , ARKPRISM constructs an information-flow subgraph G_u that connects four evidence layers: the API usage itself, the calling context that reaches it, downstream exposure points (sinks), and semantic metadata (package, permission, category). The formal definition is in Section 3. This evidence structure deliberately separates several levels: a privacy-sensitive API usage may be real even when no downstream sink is found; a call chain may be locally precise even when no trusted entry can be recovered; a taint path may indicate data reachability but not necessarily a policy violation.

2.4 Motivating Example

Listing 1 shows a simplified ArkTS pattern. The app obtains the system pasteboard, reads data in a callback, stores the text in a component state field, and later logs it from a UI interaction. A source-to-sink analyzer should report the data flow. An information-flow subgraph should additionally explain that the source belongs to the clipboard category, that it is reached from a component lifecycle or callback context, that the sink is log output, and that the source is associated with a package/namespace import.

This example illustrates why a single analysis layer is insufficient: recognition must handle the indirect receiver pb, call-chain recovery must include lifecycle and UI callback edges, taint analysis must handle callback-provided source data, and sink analysis must classify `hilog.info` as log exposure. ARKPRISM combines these layers in one report.

2.5 Recurring Failure Modes

Inspection of source/IR pairs and failed taint paths exposed six recurring classes: project-layout misses, package-migration mismatches, lost compound namespaces after IR lowering, unresolved manager receivers, missing callback edges, and lost callback-carried taint. We use these classes to define the analysis stages in Section 3 and the case-level error taxonomy in Section 5; we do not infer their corpus prevalence from the manually selected high-output projects.

```

197 1 import pasteboard from '@ohos.pasteboard';
198 2 import hilog from '@ohos.hilog';
199 3
200 4 @Entry
201 5 @Component
202 6 struct Index {
203 7   @State text: string = '';
204 8
205 9   aboutToAppear() {
206 10    let pb = pasteboard.getSystemPasteboard();
207 11    pb.getData((err, data) => {
208 12     this.text = data.getPrimaryText();
209 13    });
210 14   }
211 15
212 16   build() {
213 17    Button('debug').onClick(() => {
214 18     hilog.info(0x0, 'demo', this.text);
215 19    });
216 20   }
217 21 }

```

Listing 1. Simplified privacy-sensitive information flow in an ArkTS component.

3 Design of ARKPRISM

ARKPRISM is designed to resolve the *source identity resolution* bottleneck in ArkTS privacy analysis and to produce provenance-annotated evidence subgraphs for auditing. The design is intentionally layered. ArkAnalyzer provides the ArkTS IR, project Scene, and call-graph substrate [14]; HapFlow provides IFDS-based taint propagation over OpenHarmony programs [15]. ARKPRISM contributes the missing domain layer: package- and receiver-aware API identity resolution, bounded asynchronous carrier/context recovery, and evidence-preserving subgraph assembly with provenance tracking.

3.1 Problem Definition and Notation

Let a project be $P = \langle F, M, I, E_c \rangle$, where F is the set of ArkTS source files, M is the set of methods in ArkAnalyzer IR, I is the set of import bindings, and $E_c \subseteq M \times M$ is the initial call graph. ARKPRISM deliberately separates detector rules R_D from IFDS query rules R_T . A detector rule is

$$r_D = \langle pkg, ns, mem, direct, factory, perm, cat \rangle, \quad (1)$$

where pkg and ns identify the declaring package and namespace, mem is a method or property token, $direct \in \{true, false, null\}$ distinguishes namespace calls, receiver calls, and property reads, $factory$ is an optional set of receiver-producing methods, $perm$ is permission evidence, and cat is a privacy category. An IFDS query rule is

$$r_T = \langle sig, meta, carrier, i_{cb}, i_{src}, kind \rangle, \quad (2)$$

where sig is an SDK method signature; $meta = \langle module, ns, class, mem, params, lit \rangle$ retains the source identity needed when the frontend cannot resolve that signature; and $carrier \in \{return, arg, callback\}$ identifies the program value to seed. The source kind distinguishes typed privacy data from explicit framework inputs:

$$kind \in \{privacy_data, framework_input\}. \quad (3)$$

The optional lit component records a string-dispatched event name. For callback sources, i_{cb} is the callback-argument position and i_{src} is the sensitive callback-parameter position. A privacy-data return requires a non-void return type; a callback source requires a typed `Callback<T>` parameter

and a valid carrier index; an argument source is accepted only as an explicitly modeled framework input. Rules without this carrier contract are rejected at load time. Keeping R_D and R_T explicit is essential: a permission-bearing API operation may belong to R_D without producing privacy data, and is therefore not automatically promoted into R_T .

API identity resolution. The first task is to compute a semantic identity for each candidate API usage. We define:

$$Id(v) = \langle Pkg, Ns, Recv, Mem, Acc \rangle, \quad (4)$$

where Pkg and Ns are resolved through a namespace-constrained migration map, $Recv$ traces the receiver object back to its source via bounded backward data-flow (e.g., $cameraMgr \leftarrow getCameraMgr(ctx)$), Mem is the method or property name, and $Acc \in \{direct, assign, indirect, property\}$ classifies the access pattern. Resolution also returns a witness

$$w \in \{Namespace, IRAlias, ExactType, CallTarget, FactoryOrigin, PropertyRead\}. \quad (5)$$

A usage u is recognized when:

$$\exists s \in Stmt(P), \exists r_D \in R_D : Accept(s, r_D, I, A, w), \quad (6)$$

where A is the alias relation recovered from imports, assignments, and IR-lowered namespace-member expressions. The recognized usage set is $U_D = \{u \mid \exists s, r_D, w : Accept(s, r_D, I, A, w)\}$.

Source carriers and seeds. For a statement s matching r_T , the initial taint fact is determined by the carrier rather than by the API name alone:

$$Seed(s, r_T) = \begin{cases} lhs(s), & carrier = return, \\ arg_{i_{src}}(s), & carrier = arg, \\ param_{i_{src}}(ResolveCb(arg_{i_{cb}}(s))), & carrier = callback. \end{cases} \quad (7)$$

Property occurrences in U_D taint the right-hand-side value for detector-local tracing, but enter IFDS only when a corresponding well-formed R_T rule exists. Framework-provided Want values are query sources of kind *framework_input*, not privacy-API detector occurrences. These distinctions prevent detector coverage, privacy-data seeding, and framework-input analysis from being silently equated.

Asynchronous carrier recovery. The second task is to bind a configured asynchronous source to its consumer. We define:

$$SrcBind(call, cb, param), \quad (8)$$

where $call$ is the source API invoke statement, cb is the resolved callback method or Promise handler, and $param$ is the specific parameter through which sensitive data flows. The implementation distinguishes native IFDS propagation from bounded post-IFDS carrier recovery and from callback edges used only for call-chain context; provenance records that distinction.

Provenance-annotated subgraph. The third task is to construct, for each $u \in U_D$, an information-flow subgraph:

$$G_u = \langle V_u, E_u, \lambda_u, \omega_u \rangle, \quad (9)$$

where V_u contains API, caller, sink, and taint-fact nodes; E_u contains call, data-flow, and evidence edges, each annotated with a *provenance* label:

$$\omega(e) \in \{CG, Callback, Lifecycle, IFDS, Heuristic\}, \quad (10)$$

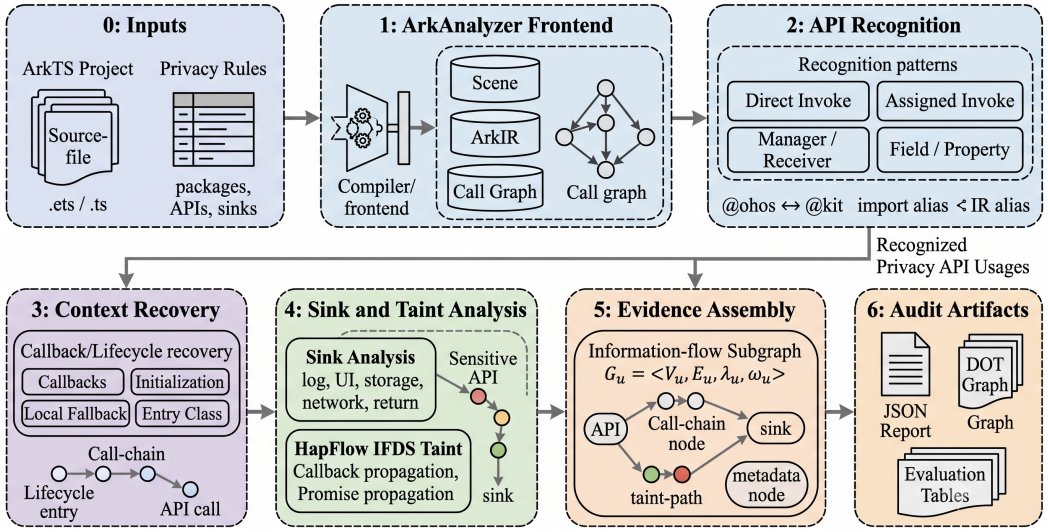


Fig. 1. ARKPRISM analysis pipeline and evidence assembly.

indicating whether the edge was derived from call-graph resolution, callback analysis, lifecycle model, IFDS propagation, or heuristic association. The provenance label is a derivation record, not a calibrated confidence: IFDS-derived edges depend on CG precision, and a CG edge from a static call may be more reliable than a lifecycle edge from over-approximate transition modeling. Auditors should interpret the label as evidence provenance rather than a global reliability ranking.

3.2 Architecture Overview

Figure 1 gives the end-to-end architecture. Project and rule inputs enter ArkAnalyzer for Scene, IR, and call-graph construction. ARKPRISM then adds three core analyses: (1) API identity resolution computing $Id(v)$, (2) bounded asynchronous carrier/context recovery computing $SrcBind(call, cb, param)$ where evidence permits, and (3) provenance-annotated subgraph assembly with $\omega(e)$ labels. Supporting mechanisms (lifecycle modeling and optional context-sensitive CG) enrich the analysis but are not the primary contributions.

3.3 Running Example: Why Direct Calls Are Insufficient

Figure 2 shows an anonymized pattern distilled from manually reviewed benchmark projects. The example is intentionally small, but each highlighted source position corresponds to a real source-level evidence class seen during manual annotation: a manager object returned by a Harmony API, a member call on that manager, a compound namespace API that may be lowered through a temporary, and a local sink that helps an auditor inspect the behavior.

A direct-call baseline that only matches `namespace.method(...)` statements can recover neither `this.calendarMgr.getCalendar(...)` nor `agent.create(...)` reliably. The former is a member call whose receiver obtains its privacy meaning from `calendarManager.getCalendarManager`; the latter is a compound namespace API whose `request.agent` prefix may be assigned to an IR temporary before `create` is invoked. ARKPRISM therefore treats API recognition as a rule- and evidence-matching problem over imports, aliases, receiver origins, and IR-lowered expressions. The resulting subgraph links the recognized API node to its lifecycle entry aboutToAppear, local sink evidence, optional taint path, and source snippet, rather than returning a flat method-name hit.

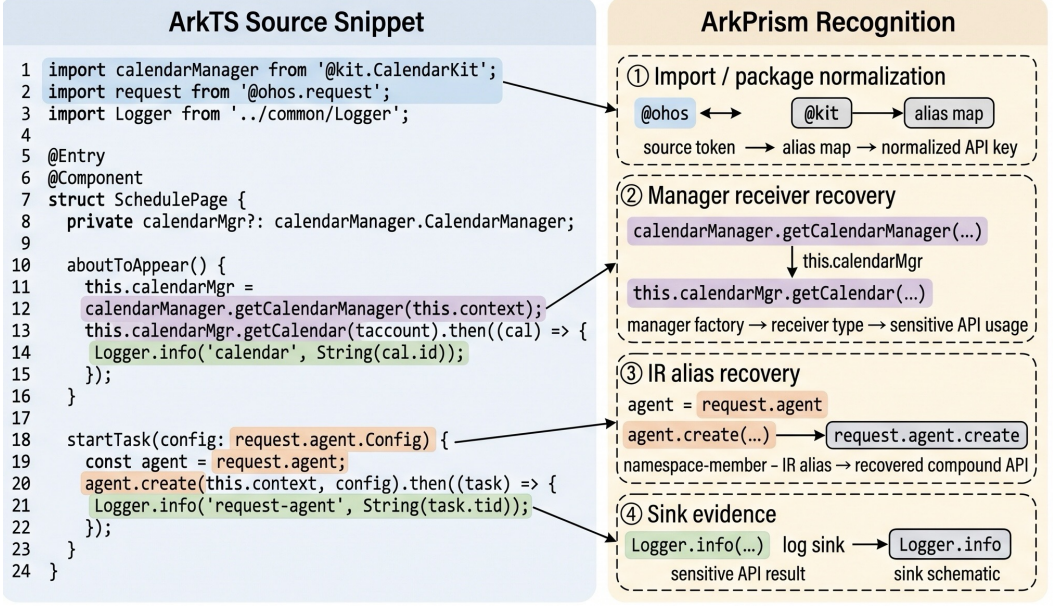


Fig. 2. Running example of API recognition and evidence construction.

3.4 Namespace-constrained Package Migration

OpenHarmony API evolution makes exact package matching brittle, but consolidated kit packages are not globally equivalent to their historical modules. We therefore define a directional migration relation over package–namespace pairs, $\mathcal{M} \subseteq (Pkg \times Ns) \times (Pkg \times Ns)$. For example, the deviceInfo export maps from `@ohos.deviceInfo` to `@kit.BasicServicesKit`, while unrelated exports in the same kit do not become compatible. A detector rule r_D and import binding i are compatible when:

$$Compat(i, r_D) \triangleq (i.pkg, i.ns) = (r_D.pkg, r_D.ns) \vee ((i.pkg, i.ns), (r_D.pkg, r_D.ns)) \in \mathcal{M}. \quad (11)$$

The analyzer also derives namespace aliases from import declarations. If a source file contains `import wifi from '@ohos.wifiManager'`, then $(wifi, wifiManager) \in A_{import}$. If it imports a kit package with a default binding, ARKPRISM records both the textual alias and the canonical namespace. Matching therefore uses a normalized API key:

$$key(u) = \langle \mu(pkg_u, ns_u), \sigma(mem_u) \rangle, \quad (12)$$

where μ applies the migration/export map and import-alias normalization to a package–namespace pair, and σ normalizes method/property spelling. Evaluation can then report both $key(u)$ and the stricter raw package signature.

3.5 Package- and Receiver-aware API Identity Resolution

HapFlow's released namespace–method loader resolves configured entries to SDK declarations. It handles directly resolvable methods, but does not encode executable property reads or retain factory-origin evidence for manager/helper receivers. These are limitations of the released loader and schema, not an impossibility result for all method-signature analyses. ARKPRISM addresses them by computing a semantic identity and an acceptance witness for each candidate usage.

3.5.1 *Identity Function.* Given a candidate statement s , ARKPRISM computes $Id(s)$ as defined in Section 3.1. The key components are $Recv(s)$ (receiver-origin tracing, detailed below) and $Acc(s)$ (access-kind classification).

The candidate predicate is:

$$\begin{aligned} Match(s, r_D, I, A) = & Compat(I_s, r_D) \\ & \wedge Id(s).Mem = r_D.mem. \end{aligned} \quad (13)$$

Package compatibility only generates candidates. Acceptance additionally requires access-specific evidence:

$$\begin{aligned} Accept(s, r_D, I, A, w) \triangleq & Match(s, r_D, I, A) \\ & \wedge Sufficient(Acc(s), w, r_D), \end{aligned} \quad (14)$$

$$\mathcal{W}_{ind} = \{ExactType, CallTarget, FactoryOrigin, Namespace\}. \quad (15)$$

$$Sufficient(a, w, r_D) = \begin{cases} w \in \{Namespace, IRAlias\}, & a \in \{direct, assign\}, \\ w = PropertyRead, & a = property, \\ w \in \mathcal{W}_{ind}, & a = indirect. \end{cases} \quad (16)$$

For indirect calls, *Namespace* means an exact namespace receiver, not merely package presence. It is disabled for generic members such as `get`, `on`, `start`, `request`, and `close`; these require type, target, or factory-origin evidence. Property acceptance requires both a property rule ($r_D.direct = null$) and an executable IR field read. Thus, package import or final-token agreement alone is never sufficient.

IFDS source-rule acceptance. The IFDS query uses SDK method signatures whenever ArkAnalyzer resolves a call target. For a call whose signature remains unknown after frontend recovery, matching falls back to the configured rule metadata rather than to method name alone. Let $Meta(r_T)$ retain the rule’s module, namespace, declaring class, member, arity, parameter types, and any string-dispatched event literal. We define:

$$Exact_T(s, r_T) \triangleq TargetSig(s) = Sig(r_T), \quad (17)$$

$$\begin{aligned} Fuzzy_T(s, r_T) \triangleq & UnknownSig(s) \wedge Mem(s) = r_T.mem \\ & \wedge ArityCompat(s, r_T) \wedge ParamCompat(s, r_T). \end{aligned} \quad (18)$$

The guard for a generic member and the guard for a string-dispatched overload are:

$$\begin{aligned} IdentityGuard(s, r_T) \triangleq & \neg Generic(r_T.mem) \\ & \vee ReceiverOrModuleWitness(s, Meta(r_T)), \end{aligned} \quad (19)$$

$$\begin{aligned} LiteralGuard(s, r_T) \triangleq & \neg HasEventLiteral(r_T) \\ & \vee ArgLiteral(s, 0) = EventLiteral(r_T). \end{aligned} \quad (20)$$

The source call is accepted iff

$$Accept_T(s, r_T) \triangleq Exact_T(s, r_T) \vee (Fuzzy_T(s, r_T) \wedge IdentityGuard(s, r_T) \wedge LiteralGuard(s, r_T)). \quad (21)$$

Consequently, an unresolved call such as `avPlayer.on('error', ...)` cannot match a configured geolocation or sensor event merely because both use `on` with the same arity. This predicate governs IFDS source seeding and is separate from the detector acceptance predicate above.

Table 1. API identity resolution examples.

Source pattern	$Id(s)$	Acc
<code>id.getOAID(cb)</code>	$\langle \text{BasicSvc}, id, \perp, \text{getOAID}, \text{direct} \rangle$	direct
<code>net=getDefaultNet()</code>	$\langle \text{NetworkKit}, net, \perp, \text{getDefaultNet}, \text{assign} \rangle$	assign
<code>mgr.getData(cb)</code>	$\langle \text{BasicSvc}, mgr, \text{getMgr}, \text{getData}, \text{indirect} \rangle$	indirect
<code>deviceInfo.brand</code>	$\langle \text{BasicSvc}, devInfo, \perp, \text{brand}, \text{property} \rangle$	property

3.5.2 Receiver-Origin Tracing. The key addition beyond the released namespace–method loader is $Recv(s)$. For a statement $s : x.m(\dots)$ where x is a local or field variable, ARKPRISM traces x backward through assignments and instance-invoke bases to find its origin, subject to a tracing bound of k_{recv} steps (default 8) to avoid unbounded backward exploration:

$$Recv(s) = \begin{cases} Ns(def), & x \leftarrow \text{invoke}(q.f()), f \in R, \\ Recv(def), & x \leftarrow y, y \text{ has known origin}, \\ TF(x), & \text{type}(x) = T, T \in SDK, \\ \perp, & \text{unresolvable in } k_{recv} \text{ steps}. \end{cases} \quad (22)$$

The first case covers the common pattern $mgr \leftarrow \text{getManager}(ctx); mgr.getData(cb)$, where the receiver mgr originates from a privacy API factory call. The second case propagates origin through intra-procedural assignment chains and instance-invoke bases. The third case (TF) uses an exactly compatible declared SDK type. Inter-method field assignments are not claimed as origin-proven unless the defining statement or an exact type/target identity is recovered; otherwise the call is rejected. This rule avoids silently treating package import plus method-name agreement as a typed call target.

3.5.3 Access-Kind Classification. Each recognized usage receives an Acc label:

- *direct*: bare invoke statement, e.g., `id.getOAID(cb)`;
- *assign*: invoke with return value assigned, e.g., `net = getDefaultNet()`;
- *indirect*: invoke on a traced receiver, e.g., `mgr.getData(cb)` where $Recv(mgr) \neq \perp$;
- *property*: field/property read, e.g., `deviceInfo.productModel`.

Table 1 illustrates how Id resolves each access kind.

IR-lowered multi-segment APIs require an additional alias relation A_{ir} . For assignments of the form $t := q.member$, ARKPRISM records $(t, q.member) \in A_{ir}$. A later statement $t.create(\dots)$ is matched as $q.member.create(\dots)$ if the composed signature appears in R_D . This is essential for APIs such as `request.agent.create`, whose namespace can disappear after lowering.

3.6 Lifecycle State Machine Model

OpenHarmony applications follow a structured lifecycle managed by the system runtime. A UIAbility transitions through creation, foreground, background, and destruction phases; custom components follow appearance, build, page visibility, and disappearance phases; UI callbacks are registered during component build and triggered by user interaction. These transitions are *implicit*—the runtime invokes lifecycle methods without visible call edges in application code—so a static analyzer that treats each lifecycle method as an independent entry point misses cross-phase data flow.

Table 2. Asynchronous carrier and context recovery rules.

Rule	Construct	Recovery relation	Output	Domain
T1	Callback source	$Resolve(a_i) \rightarrow cb.param_j$	May-path	Post-IFDS
T2	Field callback	$Read(this.f) \rightarrow cb$	Call edge	Report CG
T3	Builder option	$Arg(bld, f) \rightarrow cb$	Call edge	Report CG
T4	Promise then	$ret(src) \rightarrow then_j.param$	May-path	Post-IFDS
T5	async/await	$resolve_{arg} \rightarrow await_{res}$	May-path	Post-IFDS

ARKPRISM models the OpenHarmony application lifecycle as a three-layer state machine. Let the lifecycle model be:

$$\mathcal{L} = \langle \Phi_a, \Phi_c, \delta_a, \delta_c, \delta_{cross} \rangle, \quad (23)$$

where Φ_a is the set of ability phases (e.g., CREATING, FOREGROUNDED), Φ_c the component phases (e.g., APPEARING, BUILT), $\delta_a \subseteq \Phi_a \times \Phi_a$ the ability transition relation, $\delta_c \subseteq \Phi_c \times \Phi_c$ the component transition relation, and $\delta_{cross} \subseteq \Phi_a \times \Phi_c$ the cross-layer transition relation. Cross-layer transitions capture implicit calls such as `onForeground` $\rightarrow$ `aboutToAppear`.

The lifecycle modeler discovers ability classes by checking superclass inheritance from `UIAbility`, `Ability`, and extension ability base classes; component classes by checking superclass (`CustomComponent`, `ViewPU`), decorators (`@Component`), or the presence of two or more component lifecycle methods; and callback methods by scanning IR statements for `CommonMethod.*` invocations and extracting `FunctionType` arguments.

Lifecycle information is consumed in two distinct ways. For call-chain recovery, each transition $(m_i, m_j) \in \delta_a \cup \delta_c \cup \delta_{cross}$ becomes a provenance-labeled implicit edge. For IFDS, the model orders entry methods in a synthetic `DummyMain`: ability creation and foreground callbacks precede component initialization and UI callbacks, followed by background and destruction phases. The bounded default removes the standard back edge from the final entry to the loop header, preventing arbitrary destruction-to-creation cycles. Repeatable event classes may receive one additional explicit firing, but no unbounded loop. The IFDS solver derives interprocedural targets from the Scene and pointer information; it does not consume the separately augmented report call graph.

3.7 Asynchronous Carrier and Context Recovery

OpenHarmony privacy APIs often deliver sensitive data through asynchronous callbacks and Promise chains rather than direct return values. For example, `geoLocationManager.getCurrentLocation(callback)` passes location data to a callback parameter, while `wifiManager.getLinkedInfo().then(callback)` delivers Wi-Fi state through a Promise resolution. Standard call-to-return propagation does not by itself establish the connection between a source invocation and a callback parameter or Promise resolution, so these constructs require explicit transfer rules.

ARKPRISM formalizes carrier binding as $SrcBind(call, cb, param)$ (Section 3.1). The five recovery rules do not all modify the same analysis. T1, T4, and T5 add bounded source-to-sink outcomes after the core IFDS run when ArkIR lacks an explicit continuation edge; T2 and T3 add callback edges to the report call graph and therefore improve context recovery rather than seeding IFDS facts. Table 2 makes this execution domain explicit.

3.7.1 T1: Callback Parameter Binding. When an R_T source accepts a callback as a direct argument at position i , the binding is $SrcBind(s, cb, param_j)$ where $cb = Resolve(a_i)$ and $param_j$ is the callback

parameter selected by the rule's carrier metadata. The post-IFDS phase accepts only typed or definition-backed callbacks. Resolution applies three strategies in order:

$$Resolve(a_i) = \begin{cases} m_f, & a_i \in FuncType \wedge m_f = Lkp(a_i.sig) \\ m_c, & a_i \in ClosureType \wedge m_c = Lkp(Ext(a_i)) \\ m_l, & a_i \in Local \wedge m_l = Trc(a_i) \end{cases} \quad (24)$$

FunctionType resolution (*Lkp*) extracts the method signature from the compiler-generated function type and looks up the corresponding method in the declaring class. *ClosureType* resolution (*Ext+Lkp*) parses the closure variable name from the closure type string (e.g., closures: __lambda1) and performs a method name lookup. *Local* variable tracing (*Trc*) follows the definition chain: if a_i is assigned from a new `LambdaClass()` expression, the *ClosureType* of the constructor call provides the callback method. Example: `getData((err,data)⇒...)` resolves the arrow function to its IR method and binds `data` as the sensitive parameter.

The tracer seeds only $param_j$ selected by $r_T.i_{src}$, not every callback parameter. It propagates through ArkIR value-use edges and assignments, and accepts a sink only when a sink argument structurally depends on the current fact. IR-local names are never compared by substring, and control dependence alone does not constitute an explicit data-flow edge.

3.7.2 T2: Field-Stored Callback Binding. ArkUI components frequently store callback functions in class fields (e.g., `this.handler = () => {...}`). The callback is registered with a framework API at a later point, creating a temporal gap between definition and invocation. ARKPRISM reads the field value at the registration site and traces it to the assignment, adding $Read(this.f) \rightarrow cb$ to the report call graph. This handles the pattern where `this.onSelect` is passed to `List.onSelect(this.onSelect)` in a component's build method. T2 supplies entry/call context; it does not by itself assert a sensitive carrier or inject an IFDS fact.

3.7.3 T3: Builder-Option Callback Binding. ArkUI builder APIs accept configuration objects whose fields are callback functions (e.g., `bindPopup({builder: popupBuilder})`). T3 extracts the callback from a named option field, $Arg(bld, f) \rightarrow cb$, and adds the recovered owner-builder edge to the report call graph. The pattern is structurally distinct from T1 because the callback is nested inside a lowered object initializer rather than passed as a positional source callback. As with T2, the edge provides call context and is not an IFDS seed.

3.7.4 T4: Promise .then() Chain Binding. `Promise .then()` chains can carry a source return value through Promise resolution to a callback parameter. When the core IFDS result lacks that continuation, the bounded supplement recognizes three sub-patterns:

Direct chaining: `sourceApi(args).then(cb)` creates $SrcBind(sourceStmt, cb, param)$ with $Edge(sourceStmt, cb_{param})$.

Variable chaining: $x = sourceApi(args); x.then(cb)$ creates $Edge(sourceStmt, x_{assign}) \cup Edge(x_{assign}, cb_{param})$.

Sequential chaining: $x.then(cb_1).then(cb_2)$ creates $\bigcup_i Edge(cb_{i-1}, cb_i)$, propagating taint through the chain.

3.7.5 T5: async/await Bridge Binding. An async function that awaits a Promise-wrapped source API creates an implicit binding between the resolved value and the local variable:

$$data = await promiseMethod() \Rightarrow Edge(resolve_{arg}, await_{res}) \cup Edge(await_{res}, data_{uses}) \quad (25)$$

where $resolve_{arg}$ traces from the $resolve()$ call inside the Promise executor to the awaited result. The supplement requires a definition-backed Promise owner and separately labels the resulting path; it does not claim that the core IFDS supergraph contains the implicit executor-to-await edge.

3.7.6 Algorithm. Algorithm 1 summarizes the post-IFDS portion (T1, T4, and T5). It does not modify IFDS flow functions or the exploded supergraph. Instead, it scans configured sources whose continuation edge is absent, resolves only typed callbacks or definition-backed Promise owners, performs bounded local/interprocedural tracing, and appends separately provenance-labeled outcomes. T2 and T3 are constructed by the callback-augmented report call graph in Section 3.7; they are intentionally absent from Algorithm 1.

Algorithm 1 Post-IFDS Asynchronous Carrier Recovery

Require: Method m , IFDS source rules R_T , existing outcomes O , bounds B
Ensure: DEDUPLICATE($O \cup A$); each $a \in A$ has provenance ASYNCSUPPLEMENT

```

1:  $A \leftarrow \emptyset$ 
2: for all invoke statements  $s \in m$  do
3:    $src \leftarrow CALLSOURCE(s, R_T)$ 
4:   if  $src \neq \emptyset \wedge src.carrier = callback$  then ▷ T1
5:      $cb \leftarrow RESOLVE(s.args[src.cbIdx])$ 
6:     if  $cb \neq \emptyset \wedge src.srcIdx < |cb.params|$  then
7:        $p \leftarrow cb.params[src.srcIdx]$ 
8:        $A \leftarrow A \cup TRACEToSINK(cb, p, Path(s), B)$ 
9:     end if
10:  end if
11:  if  $s$  is a then invocation on base  $b$  then ▷ T4
12:    if  $b$  is a configured source invocation then
13:       $si \leftarrow b$ 
14:    else
15:       $si \leftarrow FINDSRCDEF(m, b)$ 
16:    end if
17:    if  $si \neq \emptyset$  then
18:       $cb \leftarrow RESOLVE(s.args[0])$ 
19:      if  $cb \neq \emptyset$  then
20:         $A \leftarrow A \cup TRACEThenToSINK(m, si, cb, B)$ 
21:      end if
22:    end if
23:  end if
24: end for
25: for all await sites  $a \in m$  with promise  $p$  do ▷ T5
26:    $ow \leftarrow FINDPROMISEOWNER(p)$ 
27:   if  $ow \neq \emptyset$  then
28:      $rf \leftarrow COLLECTRESOLVEFACTS(ow)$ 
29:     if  $rf \neq \emptyset$  then
30:        $A \leftarrow A \cup TRACEToSINK(m, a.res, Path(rf) \cup Path(a), B)$ 
31:     end if
32:   end if
33: end for
34: return DEDUPLICATE( $O \cup A$ )

```

3.8 Context-Sensitive Call Graph Construction

ARKPRISM supports k -limited context-sensitive call graph construction using ArkAnalyzer's pointer analysis infrastructure, where k controls the depth of calling context tracked per invocation. At $k = 0$ the analysis is context-insensitive; at $k \geq 1$ call sites receive distinct contexts. Context

sensitivity is an optional substrate rather than a claimed contribution; the main evaluation fixes the setting for all compared configurations instead of mixing call-graph precision with API-recognition effects.

3.9 Callback-augmented Call-chain Recovery

Given a recognized usage u inside method m_u , call-chain recovery searches a reverse graph:

$$E_r = (E_c)^{-1} \cup (E_{invoke})^{-1} \cup (E_{cb})^{-1} \cup (E_{init})^{-1}. \quad (26)$$

Here E_c is the ArkAnalyzer call graph, E_{invoke} contains IR-level call edges absent from the graph, E_{cb} contains callback edges recovered from `FunctionType/ClosureType` values, field callback assignments, and builder-option callbacks, and E_{init} links generated initialization methods to their owning class or component context. The recovered chain set is:

$$C_u = \{ \pi = (m_0, \dots, m_u) \mid m_0 \in \text{Entry} \cup \text{Init} \cup \text{Unknown}, \\ (m_i, m_{i+1}) \in E_r^{-1} \}. \quad (27)$$

Each chain receives an entry-evidence class:

$$\kappa(\pi) = \begin{cases} \text{framework}, & m_0 \in \text{Entry} \\ \text{initialization}, & m_0 \in \text{Init} \\ \text{local}, & \pi = (m_u) \text{ and no predecessor is found} \\ \text{unknown}, & \text{otherwise.} \end{cases} \quad (28)$$

ARKPRISM does not collapse these classes into a single coverage number. A local fallback chain is useful evidence that the API usage exists in a method, but it is not a recovered framework-entry path. This distinction is preserved in the report and in the evaluation tables.

3.10 Sink and Taint Attachment

For each detector occurrence $u \in U_D$, local tracing extracts statements $K_D(u)$ that may expose privacy-related data through logging, UI display, storage, network transfer, or data return. Sink resolution requires an exact configured method token and compatible receiver/namespace evidence; in particular, substring agreement such as `set` versus `setData` is rejected.

The IFDS query is configured independently by R_T and its sink set K_T . Let S_P and S_F be source statements whose rules have kind *privacy_data* and *framework_input*, respectively. The solver computes

$$T_I \subseteq S_T \times K_T \times \text{Path}, \quad (29)$$

where $S_T = S_P \uplus S_F$ is the disjoint typed-source universe resolved from R_T . Each tuple $\langle s, k, \rho \rangle$ means that the carrier seeded at s may reach configured sink k along path ρ . The bounded post-IFDS rules produce a separate set

$$T_A \subseteq S_T \times K_T \times \text{Path}, \quad (30)$$

and the report exposes $T_Q = T_I \cup T_A$, stratified as

$$T_P = \{q \in T_Q \mid \text{src}(q) \in S_P\}, \quad T_F = \{q \in T_Q \mid \text{src}(q) \in S_F\}. \quad (31)$$

For duplicate endpoint/path outcomes, path provenance is retained as

$$\tau(q) \in \{ifds, async_supplement, both\}. \quad (32)$$

A partial linkage relation

$$L \subseteq U_D \times T_P \quad (33)$$

is created only when project, normalized source location, and compatible API identity agree. Framework-input paths are never linked to privacy-API detector occurrences. Linked privacy-data paths are attached to the corresponding G_u ; unlinked configured-query paths remain project-level taint evidence rather than being forced onto a detector occurrence. Consequently, detector-local sink-positive projects and configured-query-path-positive projects are not set-inclusion counts. The corpus evaluation reports both evidence universes and both source kinds separately.

3.10.1 Provenance Annotation. Each edge $e \in E_u$ carries a provenance label $\omega(e) \in \{IFDS, Callback, Lifecycle, CG, Heuristic\}$ (Section 3.1), recording *how* the edge was derived: *IFDS* for solver-propagated facts; *Callback* for a report-CG callback edge or post-IFDS asynchronous supplement; *Lifecycle* for implicit phase transitions; *CG* for call-graph edges; and *Heuristic* for name-based inferences without formal proof. The record also identifies its producing phase, so a callback supplement is not mislabeled as an IFDS proof. Provenance is not a calibrated confidence score: an IFDS edge depends on call-graph precision, and a lifecycle edge may over-approximate feasible order. We therefore evaluate flow classification separately from evidence availability and do not convert provenance labels into an unsupported probability of correctness.

3.11 Analysis Contract and Soundness Envelope

ARKPRISM is a soundy audit analysis, not a proof system. Its recognition claim is relative to (i) ArkAnalyzer successfully lowering a source construct to IR, (ii) the API appearing in the fixed rule set, and (iii) package compatibility plus the required namespace, receiver, origin, or target-signature evidence satisfying *Accept*. Native code, dynamic loading, reflection-like runtime dispatch, and APIs absent from the rule set lie outside this contract. A recognized API is evidence of use, not evidence of a privacy violation.

IFDS and asynchronous-supplement results are may-flows under their respective source/sink and reachability models. The report preserves their producer and does not convert a detector occurrence into an IFDS source without a source-endpoint link. Every run records one of three solver states: complete, resource-bounded, or failed. Resource-bounded and failed cases are never reclassified as zero-flow applications. Edge deduplication is scoped by IR statement and value identity; textual equality alone is insufficient because same-named locals in different methods are distinct facts. These requirements are regression-tested on HapBench positive and negative cases and are also enforced by the isolated-process corpus runner.

3.12 Corpus-scale Hardening

Corpus-scale execution requires additional engineering. SDK files are loaded lazily into the Scene only when taint analysis starts, avoiding SDK-induced type instability during initial project construction. Source and sink JSON rules expand SDK-path placeholders at runtime. IFDS budgets are explicit. Each project runs in a fresh process because retaining SDK-augmented scenes across projects causes cumulative heap growth; per-project logs and status files make timeouts and failures visible. If pointer analysis fails, the runner records the warning and continues only with evidence whose producing phase completed. These choices prevent incomplete analysis states from silently becoming negative results.

The complete pipeline follows a fixed order: Scene construction, lifecycle modeling, report call-graph construction, import/alias extraction, API identity resolution via *Accept*, callback-augmented chain recovery, detector-local sink tracing, IFDS taint analysis from the lifecycle-aware *DummyMain*, bounded asynchronous supplementation, endpoint linkage, and subgraph assembly. A key property is that ARKPRISM never treats a missing layer as proof of absence: if a call-chain predecessor is absent, the usage still becomes a local evidence chain; if a taint path has no compatible detector endpoint, both records remain visible but unlinked.

4 Implementation

ARKPRISM is implemented in TypeScript and runs as a command-line analyzer. It reuses ArkAnalyzer's Scene, IR, and call-graph abstractions, and integrates HapFlow through a runner module.

Rule bases. The release configuration in `sensitive_apis.json` contains 58 nonempty system-package groups and 716 detector entries (665 unique namespace-member keys): 467 direct, 238 manager/receiver, and 11 property rules. Each entry records namespace, method/property, permission, profiling category, direct/indirect/property mode, and optional receiver factories. A deterministic pre-release audit removed 104 byte-equivalent duplicate entries and six legacy duplicates that differed only by an empty class field; semantic rule-set equality is regression-tested. A domain audit then corrects 67 inherited profiling labels (e.g., account, NFC, cellular, MDM, notification, and authentication entries previously placed in a Bluetooth catch-all), yielding 38 explicit categories and 74 distinct permission labels.

Detector entries are not automatically IFDS sources. An earlier generated configuration had promoted 655 untyped detector operations, including setters and publication/network actions, to return sources without a return type or sensitive-value rationale. A second SDK-level audit found 29 return rules with non-concrete carriers: stale any declarations, a `Promise<void>` operation, properties represented as methods, wrong module owners, and APIs absent from API 20. We replaced supported methods with exact API-20 overloads, retained property reads only in the detector until field seeding is modeled, and removed one HMS rule because that external SDK is not loaded by the frozen analysis. The resulting IFDS inventory contains 257 privacy-data rules and two separately loaded *framework_input* lifecycle rules.

The resolver treats an explicit parameter array, including the empty array, as an exact arity constraint and compares normalized parameter types and literal discriminators. This prevents a zero-argument Promise rule from absorbing a callback overload and prevents event-specific on/off rules from matching another event with the same parameter names. Module, namespace, and class are hard owner constraints: if an explicitly named class is unavailable in the loaded SDK, resolution fails instead of falling back to another class with the same member name. Against the frozen API-20 SDK, all 257 privacy-data rules resolve to exactly one declaration: there are no unresolved or ambiguous rules, signature collisions, or carrier-type mismatches. The two framework-input rules likewise resolve uniquely under both the API-17 benchmark SDK and API-20 corpus SDK. The API-17 compatibility audit resolves 249 of the 257 privacy-data rules and records the eight version-incompatible rules without substituting a different owner. The loader also rejects a missing source kind, non-concrete return carrier, callback type, or carrier index. The structural audit reports no malformed detector, source, or sink entries and 29 configured sinks. A namespace-constrained migration table maps historical modules to exports of consolidated `@kit.*` packages; it is not treated as whole-package equivalence. Detector rules, privacy-data sources, framework inputs, SDK-resolution records, and hashes are retained separately.

Detector. The API detector operates per Ark file: it obtains system imports, expands package aliases, divides rules into direct, indirect, and constant-like sets, and scans every method CFG. Method matching normalizes `Class.method`, `method()`, and event-style signatures while

preserving namespace context. Package-compatible rules form the candidate set; indirect calls are then accepted only with an exact SDK receiver type, exact declaring-class identity from the call-target signature, an intra-procedural definition/factory-origin chain (bounded at eight definitions), or an exact namespace receiver for a non-generic member. The selected evidence is stored in `matchEvidence`; package-only and ambiguous method-only matches are rejected. For IR-lowered multi-segment APIs (e.g., `request.agent.create`), the detector records namespace-member aliases when ArkAnalyzer lowers the namespace prefix into a temporary.

Call-chain tracer. The tracer builds an enhanced reverse call map from ArkAnalyzer call-graph edges plus supplementary edges recovered from `FunctionType/ClosureType` signatures, class-field callback assignments, and ArkUI builder-option callbacks. Entry methods are classified through lifecycle and callback naming conventions; if no upstream entry is found, the tracer emits a local fallback chain.

Detector-local sink analysis. For each detector occurrence, the sink analyzer follows local and callback-related values and classifies observations into log, UI display, storage, network, data return, intent, and share categories. A semantic-enrichment pass adds guard conditions, try-catch context, and loop evidence. These observations are not the IFDS sink universe.

IFDS runner and asynchronous supplement. The runner lazily loads SDK files, creates the lifecycle-aware `DummyMain` entry (Section 3.6), attempts pointer analysis, and loads the independent IFDS source/sink query. The solver derives call targets from Scene and pointer-analysis evidence, including receiver-refined indirect targets; it does not reuse the separately augmented report call graph. Unknown-signature source matching retains the rule’s module, namespace, class, arity, and parameter metadata. Generic members such as `on`, `get`, and `create` require receiver/module evidence; a string-dispatched overload such as `on('locationChange', ...)` must additionally match the configured event literal. Thus, an unrelated `avPlayer.on('error', ...)` cannot inherit a geolocation or sensor source solely from the member name and arity.

IFDS edge keys combine IR object identity with access-path state rather than statement text, preventing distinct same-named locals from being merged. The solver runs all sources together by default; batching is available only as an explicit resource fallback. After IFDS terminates, a separately bounded callback/Promise scan recovers typed callback, then, and `await` paths that are absent from the supergraph. It seeds the configured callback-parameter index and uses structured `Value.getUses/ValueEqual` dependencies rather than textual IR-local matching; in particular, temporary %8 does not match %85. Outcomes are deduplicated only when their tainted values and complete statement paths agree, then merged with, but not relabeled as, solver outcomes. Unknown-signature sinks likewise require method and receiver/namespace agreement, and failure or budget exhaustion is recorded rather than emitted as an empty negative result.

Evidence linkage. Every path records whether its seed is *privacy_data* or *framework_input*. Subgraph assembly considers only privacy-data paths, and joins one to a detector occurrence only when project, normalized source location, and compatible API identity agree. Detector-local sink observations, unlinked privacy-data paths, and framework-input paths remain separate records. This explicit partial join prevents aggregate IFDS paths from being interpreted as downstream sinks of every detector occurrence.

Lifecycle modeler and context-sensitive CG. The lifecycle modeler builds $\mathcal{L}$ (Section 3.6) and orders ability, component, and event entries in `DummyMain`. The default removes unbounded lifecycle back edges while allowing an explicit second firing for selected repeatable events. Separately, the report call graph augments RTA/CHA with provenance-labeled lifecycle transitions. The context-sensitive CG module wraps ArkAnalyzer’s pointer analysis with k -limited context sensitivity (Section 3.8).

5 Evaluation

We evaluate four questions: **RQ1** How accurately does ARKPRISM resolve privacy-API identities in real projects, and which evidence forms produce that accuracy? **RQ2** How accurately do ARKPRISM and the author-released HapFlow artifact classify end-to-end flows? **RQ3** Which recovery mechanisms account for ARKPRISM's behavior, and what precision–recall trade-offs do they introduce? **RQ4** What privacy-evidence patterns and scalability characteristics appear in a large OpenHarmony corpus? We separate these claims: source-first and controlled identity benchmarks support API recognition, HapBench and ArkAsyncBench support flow classification and mechanism attribution, and the corpus supports prevalence and scalability rather than ground-truth accuracy.

5.1 Experimental Design

Implementations and configuration. We evaluate the author-released HapFlow artifact and the released ARKPRISM implementation. Both consume ArkAnalyzer IR and the same HapBench source–sink query. For artifact comparability, HapBench uses the OpenHarmony API-17 SDK (version 5.0.5.165) bundled with the HapFlow release; the corpus experiment uses the independently installed API-20 SDK (version 6.0.0.47). SDK fallback is disabled, and both directory fingerprints are retained. The large-corpus runner launches one analyzer process per project, with two concurrent workers, so that heap state and failures are isolated across projects. HapBench uses the bounded lifecycle policy and unbatched IFDS solver described in Section 4. The main configuration enables the supplementary callback scan; a paired callback-off run isolates its contribution from the IR/IFDS analysis.

The final corpus artifact is produced by one process-isolated execution of the final build over the 1,014 retained projects; no change-impact overlay enters the reported aggregates. The run manifest records the analyzer revision and build hash, rule hashes, SDK fingerprint, feature flags, resource limits, and excluded partial project. Every retained log identifies the API-20 SDK, completes PTA, and reports a single unbatched IFDS run without budget exhaustion.

Evaluation units. We use six complementary evidence sets. *ArkSourceFirst60* is an output-independent real-project benchmark. Before running the evaluated detector, we deterministically sample 60 projects across small, medium, and large source-tree strata: 30 contain an import compatible with the frozen API rules and 30 do not. A structured AST queue locates executable member accesses but assigns no label. Manual source review then accepts or rejects all 101 candidates using imports, receiver definitions, factory origins, SDK types, and access context, and inspects configured imports in candidate-free projects. The resulting occurrence-level gold set contains 90 accepted accesses in 16 projects, 11 high-confusion rejected candidates, and 44 projects with no configured sensitive access. Every gold record stores project, file hash, exact line and column, canonical API identity, access kind, receiver/source expression, category, permission, and the imports used during review.

ArkIdentityBench contains 32 positive and 32 matched-negative cases spanning namespace calls, properties, typed and constructed receivers, factory-returned managers, compound namespaces, aliases, and historical-to-consolidated package migration. Its progression baselines successively add lexical, import, declared-receiver, and factory evidence under the same case oracle. It isolates identity-resolution mechanisms from downstream call-graph and taint behavior.

The *120-project high-output audit* complements source-first coverage with a stress population rich in reported identities. It contains 666 manually confirmed project–API keys backed by executable source locations. Because the ranking begins from ARKPRISM output and canonicalizes repeated occurrences to project–API keys, this secondary artifact supports stress precision rather than occurrence recall.

HapBench is the complete 67-case suite distributed with HapFlow: 53 expected-flow and 14 expected-no-flow cases in seven categories. We reconstruct the oracle from each case’s inline source, sink, and leak/no-leak annotations and cross-check the category totals against the paper. We publish the case-level oracle and preserve disagreements with the executable artifact.

ArkAsyncBench is a source-first controlled benchmark fixed before executing the evaluated configurations. Its 48 cases use three configured privacy-data APIs—advertising identifier, SIM account information, and reverse geocoding—under callback (T1), Promise then (T4), and await (T5) constructs. The 24 positive cases pass the configured carrier, a carrier property, or a carrier alias to `console.info`; the 24 matched negatives pass only an error, constant, rejection value, ignored result, or similarly named independent local. The oracle is therefore defined by explicit source-level value dependence rather than by either configuration’s output. This benchmark isolates the implemented asynchronous rules; it does not claim completeness over all event-loop or Promise semantics.

The *large corpus* begins with 1,015 OpenHarmony/ArkTS project directories. Of these, 1,014 satisfy the complete-analysis criterion and cover 31,205 files and 233,894 methods. One project reaches the unbatched IFDS time bound after successful PTA and is retained as a partial artifact but excluded from aggregate analysis. A token-based redundancy analysis removes comments and trivia but preserves identifiers and literals, then computes SHA-256 file fingerprints, exact project-clone clusters, and near-clone clusters. This evidence supports prevalence, concentration, robustness, and cost analyses rather than ground-truth accuracy.

Metrics and statistics. At the identity-occurrence level, equality requires project, normalized file, exact line, canonical package/namespace/member, and access kind; a nearby call to the same API cannot mask a missed occurrence. Project-level identity results count both API-positive and confirmed-zero projects. At the HapBench and ArkAsyncBench case levels, a positive prediction means that at least one privacy-data source-to-sink path is reported; framework-input paths cannot satisfy either oracle. We report precision, recall, specificity, balanced accuracy, F1, and MCC, with Wilson intervals for binomial metrics and exact two-sided McNemar tests for paired outcomes. For the high-output audit, we retain project-key and project-level Wilson intervals but do not reinterpret its canonicalized keys as occurrence recall.

5.2 RQ1: Privacy-API Identity Resolution

Source-first real-project accuracy. ARKPRISM reports exactly the 90 gold occurrences in Ark-SourceFirst60: TP/FP/FN= 90/0/0, yielding 100% observed occurrence precision, recall, and F1 (95% Wilson CI for precision and recall: 95.91–100%). At project level it classifies all 16 positive and 44 zero projects correctly. Exact-location matching matters here: IR statement locations are post-resolved to the corresponding executable AST access, preventing one detected access from covering another same-API access elsewhere in the file.

The gold set spans 44 namespace properties, 26 namespace calls, 11 typed receivers, four constructed receivers, three manager receivers, and two factory receivers. Thus, 20/90 occurrences require receiver evidence unavailable from the accepted line’s bare namespace syntax, while nearly half are executable properties rather than calls. The 11 rejected candidates exercise the complementary precision boundary: same-name application wrappers, collection/controller operations, and library calls such as `create`, `on`, `off`, and `getData` are excluded when their package and receiver origins do not establish a configured identity.

Mechanism progression. Table 3 quantifies how evidence changes the decision boundary on ArkIdentityBench. A lexical matcher finds every positive but also every matched collision. Import evidence removes those collisions yet reaches only 17/32 positives; declared receiver types recover

nine more, and factory tracing recovers another two. The complete resolver adds constructed receivers, executable properties, compound aliases, and namespace-constrained migration, classifying all 64 cases correctly.

Table 3. Identity-resolution progression on ArkIdentityBench.

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1
Lexical token	32	0	32	0	50.00	100.00	0.00	66.67
Import-aware namespace	17	32	0	15	100.00	53.13	100.00	69.39
Declared-receiver AST	26	32	0	6	100.00	81.25	100.00	89.66
Factory-aware AST	28	32	0	4	100.00	87.50	100.00	93.33
ARKPRISM	32	32	0	0	100.00	100.00	100.00	100.00

High-output stress audit. All 666 reviewed project-API keys in the 120-project stress population have an executable supporting access (100% observed project-key precision; 95% Wilson lower bound 99.43%). All 120 projects contain only confirmed keys (project-level lower bound 96.90%). Its evidence tags include 558 namespace-qualified accesses, 102 member/receiver accesses, five executable template accesses, and one compound-qualified call. This distinct aggregation unit strengthens precision evidence under dense output without replacing the occurrence-level source-first benchmark.

Figure 2 illustrates the evidence composition on a compact example: `deviceInfo.brand` is an executable property, whereas `helper.createAsset()` requires a definition chain to `photoAccessHelper.getPhotoAccessHelper(...)`. At the schema level, the released HapFlow loader has no field/property source form or detector-style factory witness; the executable, same-SDK end-to-end comparison is reported separately in RQ2.

Answer to RQ1. On the source-first sample, ARKPRISM resolves all 90 manually accepted occurrences at exact locations and rejects all candidate collisions, including 44 zero projects. Controlled progression shows that its gain comes from composing package, receiver, factory, property, and IR identity evidence rather than relaxing member-name matching. The 120-project stress audit supplies consistent high-volume precision evidence over 666 additional project-API keys.

5.3 RQ2: End-to-End Flow Classification

Table 4 reports the complete artifact reproduction. Both tools finish all 67 cases. ARKPRISM classifies 64 correctly (50 TP, 14 TN), compared with 61 (51 TP, 10 TN) for HapFlow. It eliminates all four false positives while retaining 50 of 53 positives, yielding 97.09% F1, 97.17% balanced accuracy, and 0.881 MCC, versus 94.44%, 83.83%, and 0.717. Its precision is 100% (95% CI: 92.9–100%), recall is 94.34% (84.6–98.1%), and specificity is 100% (78.5–100%).

Table 4. Complete HapBench reproduction (53 positive and 14 negative cases).

Tool	TP	TN	FP	FN	Precision	Recall	Specificity	F1	BAcc.	MCC
HapFlow [15]	51	10	4	2	92.73%	96.23%	71.43%	94.44%	83.83%	0.717
ARKPRISM	50	14	0	3	100.00%	94.34%	100.00%	97.09%	97.17%	0.881

ARKPRISM alone is correct on six cases and HapFlow alone on three; the exact McNemar test gives $p = 0.508$. Accordingly, our comparative claim is the observed improvement in precision,

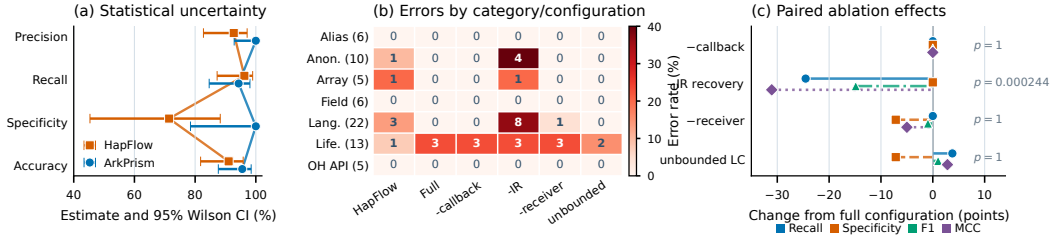


Fig. 3. Statistical and configuration-level diagnosis on HapBench. (a) Point estimates and 95% Wilson intervals. (b) Category–configuration error rates, with cells reporting error counts. (c) Metric changes relative to the full configuration and exact paired McNemar tests.

specificity, and class balance on this suite. Figure 3(a) reports Wilson uncertainty for the principal rates, while panel (b) localizes errors across all seven benchmark categories and six evaluated configurations. Macro-category F1 is 97.96% for ARKPRISM and 95.73% for HapFlow. ARKPRISM obtains 100% F1 in six of seven categories, with 85.71% in lifecycle cases.

Error mechanisms. HapFlow’s two false negatives expose omitted anonymous-instance initialization and exceptional-flow recovery. Its four false positives arise from collapsing distinct array indices, imprecise virtual dispatch in two cases, and a lifecycle edge that makes UnreachableFlow reachable. ARKPRISM repairs these mechanisms through constructor-to-instance-initializer edges, lexical-environment and exception propagation, constant-index access paths, receiver-refined dispatch, and bounded lifecycle transitions.

ARKPRISM’s three false negatives are all lifecycle cases. Source inspection reveals that they also stress the oracle semantics. In ActivityLifecycle4, a child override does not invoke the parent source-bearing onCreate; BackupExtensionAbility requires an instance field to survive into a later restore instance; and Button1 requires static state written by a click callback to reach a later ability creation. We retain the published/source-annotated positive labels, report all three as false negatives, and do not add unconstrained edges solely to match them.

Because these labels depend on object and event-order assumptions, Appendix A.2 repeats the comparison under a strict executed-order contract that relabels only these three cases. Under this secondary, post-hoc sensitivity oracle, ARKPRISM classifies all 67 cases correctly, whereas HapFlow classifies 58 (48 TP, 10 TN, 7 FP, and 2 FN); the exact paired test gives $p = 0.00390625$. We retain the published labels as the primary result and use this sensitivity analysis only to expose how lifecycle semantics affect the comparison.

Answer to RQ2. On the complete executable suite, ARKPRISM attains 100% precision, 94.34% recall, and 97.09% F1 with no false positives. Its principal empirical advantage over the released HapFlow artifact is negative-case discrimination, reflected in higher specificity, balanced accuracy, and MCC.

5.4 RQ3: Mechanism Contribution and Trade-offs

We rerun all 67 cases after disabling the supplementary callback scan, disabling IR recovery, disabling receiver refinement, or replacing the bounded lifecycle model with an unrestricted lifecycle cycle. Each ablation changes one mechanism and uses the same oracle and source–sink query. Table 5 reports the aggregate classifications; Figure 3(b–c) localizes their category-level errors and paired metric effects.

Table 5. HapBench mechanism ablations.

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	BAcc	F1
ARKPRISM	50	14	0	3	100.00	94.34	100.00	97.17	97.09
–callback supplement	50	14	0	3	100.00	94.34	100.00	97.17	97.09
–IR recovery	37	14	0	16	100.00	69.81	100.00	84.91	82.22
–receiver refinement	50	13	1	3	98.04	94.34	92.86	93.60	96.15
Unbounded lifecycle	52	13	1	1	98.11	98.11	92.86	95.49	98.11

HapBench isolates core recovery mechanisms. The complete suite attributes its configuration differences to IR, dispatch, and lifecycle recovery. Its callback constructs already have explicit core-IFDS edges, so disabling the supplement preserves the same 50 positive cases and 53 unique paths. We therefore evaluate incremental asynchronous coverage with a benchmark constructed around continuation gaps rather than assigning that role to HapBench.

ArkAsyncBench exposes complementary coverage. Table 6 separates paths supported by core IFDS from those reconstructed by the asynchronous supplement. The full configuration classifies all 24 positive and 24 matched-negative cases correctly. T1 callbacks and T5 await bridges are independently supported by both analyses, while all nine T4 then positives are supplement-exclusive. Removing the supplement therefore preserves 15 core-supported positives but misses the nine T4 cases (exact McNemar $p = 0.00390625$). This decomposition turns provenance into a mechanism check: BOTH marks corroborated T1/T5 paths, and ASYNCSUPPLEMENT identifies the unique T4 increment.

Table 6. Complementary asynchronous-carrier coverage.

Construct	Pos.	Full	Core	Suppl.	Exclusive	Neg. rejected	Real paths
T1 callback	9	9	9	9	0	9/9	1
T4 then	9	9	0	9	9	9/9	1
T5 await	6	6	6	6	0	6/6	6
Total	24	24	15	24	9	24/24	8

Real-project asynchronous diagnostic. We rerun 26 callback-heavy real projects with the final typed-source configuration. It produces 16 privacy-data paths in six projects, including eight asynchronous-only paths; twelve framework-input paths remain a separate source universe. Manual source/IR review confirms all eight structured dependencies (95% Wilson lower bound 67.56%): one T1 callback carries metadata.genre, one T4 Promise carries an advertising identifier through a field assignment, and six T5 paths carry awaited media metadata or reverse-geocoding results into log arguments. These real-code instances align each controlled construct with an executable application pattern.

IR recovery is necessary. Removing constructor/initializer, lexical-capture, exception, array, and alias recovery creates 13 additional false negatives. F1 falls by 14.87 points and MCC by 0.310. The full configuration is exclusively correct on 13 cases and never exclusively wrong relative to this ablation (exact McNemar $p = 0.000244$). This remains significant under Bonferroni correction for the four pre-specified ablations ($\alpha = 0.0125$). The affected cases span anonymous constructs, closures, exception handling, array copying, multi-field access, and virtual dispatch, showing that the gain is not concentrated in a single hand-written pattern.

Receiver refinement protects specificity. Without receiver-sensitive target filtering, VirtualDispatch3 becomes a false positive. Recall is unchanged, but specificity falls from 100% to 92.86% and MCC from 0.881 to 0.831. This single controlled counterexample is important for privacy analysis: resolving more targets is not automatically an improvement when unrelated implementations introduce spurious source-to-sink paths.

Lifecycle bounds encode a precision policy. The unrestricted lifecycle variant accepts two additional cross-event cases, but also reintroduces the explicit UnreachableFlow false positive. Because F1 ignores true negatives in a 53-positive/14-negative suite, its raw F1 rises to 98.11% even as specificity falls to 92.86% and balanced accuracy to 95.49%; the bounded configuration retains 100% specificity and 97.17% balanced accuracy. Under the strict executed-order sensitivity oracle in Appendix A.2, bounded ARKPRISM classifies 67/67 cases, whereas unbounded lifecycle creates three cross-event false positives and classifies 64/67. The bounded default therefore enforces the paper’s precision-first reachability contract.

Answer to RQ3. IR recovery provides the largest statistically supported HapBench contribution, receiver refinement removes a concrete dispatch false positive, and bounded lifecycle preserves perfect negative-case discrimination. On ArkAsyncBench, the asynchronous supplement independently supports every positive carrier, uniquely recovers all nine T4 Promise-then cases, and introduces no negative-case errors; eight corresponding paths are confirmed in real projects.

5.5 RQ4: Large-Corpus Characterization

The retained analysis covers 31,205 ArkTS/TypeScript files and 233,894 methods in 1,014 projects. It identifies 1,896 detector-recognized privacy-API occurrences in 273 projects, 1,542 detector-local sink observations in 185 projects, and 1,103 configured-query source-to-sink may-paths in 257 projects. The path total combines core solver outcomes and separately provenance-labeled asynchronous supplements. Detector-local sinks are attached to detector usages; configured-query paths use hapflow_sources.json/hapflow_sinks.json. They are therefore separate evidence universes, and the 257 versus 185 project counts do not assert that a path exists without a configured sink endpoint. The final-run audit verifies valid path endpoints, provenance tags, and detector-to-path link indices. These records are *audit evidence*, not confirmed leaks.

Table 7. Large-corpus privacy evidence.

Measure	Count	Project rate	Interpretation
Detector-recognized API occurrences	1,896	26.92%	Accepted identity witness
Detector-local sink observations	1,542	18.24%	Local/callback sink tracing
Configured-query may-paths	1,103	25.35%	Solver/supplement provenance
Analyzed files/methods	31,205/233,894	–	Frontend analysis scope

Concentration and category structure. The distribution is long-tailed: the median project has no recognized privacy API, while the top 10 projects account for 26.42% of occurrences and the top 50 account for 59.44%. The API-occurrence Gini coefficient is 0.886; the corresponding HHI is 0.0121, equivalent to 82.9 equally sized contributing projects. Hardware identity, network connectivity, screen/display identity, account data, and clipboard data contribute 18.62%, 14.40%, 11.71%, 9.49%, and 5.96%, respectively. Project scale is associated with evidence volume (Spearman $\rho = 0.461$ between method count and API occurrences), but the dispersion in Figure 4 shows that size alone is not a reliable privacy proxy.

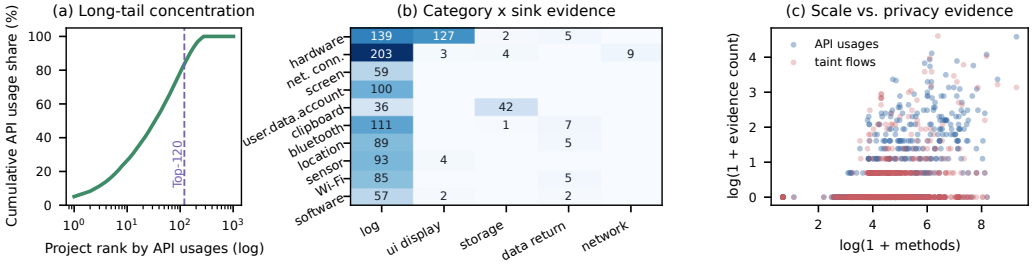


Fig. 4. Detector/configured-query evidence concentration, detector-local category-sink structure, and project-scale relationships in the large corpus.

Destinations and path structure. Logging dominates the 1,542 detector-local sink observations (1,272; 82.49%), followed by UI display (136; 8.82%), storage (93; 6.03%), data return (32; 2.08%), and network transmission (9; 0.58%). This distribution characterizes detector-local audit destinations and is not used to label the endpoints of all 1,103 configured-query paths. Call chains have median length 1 and P95 length 4; configured-query paths have median length 5, P95 length 9, and maximum length 33.

Solver integrity and cost. All 1,014 retained projects finish with SUCCESS; no retained log contains a PTA fallback, IFDS batching, or budget exhaustion. The median project processes 837 IFDS edges (P95 13,857; maximum 778,986). Exact flow deduplication reduces 1,549 raw paths to 1,103 unique paths. Median per-project wall time is 16.4 s, P95 is 21.2 s, and the 351.3 s maximum belongs to the 10,757-method outlier. The moderate method-count/runtime association (Spearman $\rho = 0.393$) reflects both project scale and fixed frontend/SDK costs.

Clone sensitivity. The token profile over the 1,015 candidate directories contains 32,233 source files, 21,663 unique token fingerprints, nine exact-clone clusters, and eleven near-clone clusters. Over the 1,014 completed analyses, counting one representative per exact-clone cluster changes API-, sink-, and path-positive project rates from 26.923%, 18.245%, and 25.345% to 27.236%, 18.392%, and 25.628%, respectively. Using one representative per near-clone cluster yields 27.090%, 18.328%, and 25.680%. The maximum absolute shift is 0.335 percentage points. The aggregate prevalence therefore does not arise from repeatedly counting a small number of copied demonstrations, although shared framework code can still influence individual category frequencies.

Answer to RQ4. Privacy evidence is sparse across projects but concentrated within a small high-output tail. Local logging is the dominant destination, project size only partially explains evidence volume, and clone-aware resampling leaves project-level rates essentially unchanged.

6 Discussion

6.1 What the Evidence Supports

The evaluation assigns each claim to an oracle with the matching unit. ArkSourceFirst60 supports exact-location occurrence precision and recall on an output-independent, size/import-stratified real-project sample that includes 44 confirmed zero projects. ArkIdentityBench isolates recognition mechanisms under matched positive and negative patterns. HapBench and ArkAsyncBench support case-level flow classification and asynchronous mechanism attribution, respectively. The 120-project stress audit tests dense reported identities, while the 1,014-project corpus supports prevalence, robustness, and scalability observations rather than ground-truth accuracy. Within the corpus, detector-local sinks, typed privacy-data paths, and framework-input paths remain separate evidence

universes; a conservative endpoint relation links detector usages only to compatible privacy-data sources.

The resulting subgraphs should also be interpreted as audit evidence. A may-flow to a log or UI operation is not automatically a privacy violation, and a local fallback chain is not a complete lifecycle path. Provenance labels expose whether an edge came from IFDS, callback supplementation, lifecycle modeling, or report-CG recovery, but they do not assign a probability that the edge is feasible. The final-run audit validates schema integrity, endpoint resolution, provenance presence, and link indices; it is not a user study or a semantic proof of every subgraph edge.

6.2 Threats to Validity

Benchmark validity. HapBench is the only public, executable OpenHarmony taint benchmark available to us. Its 67 cases provide controlled positive and negative examples, but remain small and often synthetic. We reconstruct its case oracle from the artifact’s inline leak/no leak annotations and cross-check category totals against the HapFlow paper. The outputs produced by the released executable artifact do not exactly reproduce the paper’s aggregate table. We therefore publish every case label and actual output, and retain the source annotations rather than modifying the oracle to match a reported aggregate.

Source-benchmark validity. ArkSourceFirst60 is frozen from project scale and configured-import strata without reading detector reports. Candidate generation is a navigation aid: every accepted and rejected item is decided from source context, and exact file hashes and locations make those decisions auditable. This design directly includes API-positive and confirmed-zero projects, but its 60 projects and author-reviewed labels delimit the population to which the observed 90/90 result applies. The separate Top-120 audit is intentionally output-ranked and therefore contributes high-volume precision evidence. Together, the two sets expose different failure modes rather than allowing one selection strategy to stand for both accuracy and stress testing.

Corpus validity. The corpus contains applications, libraries, and many capability demonstrations. Of 1,015 candidate directories, 1,014 satisfy the complete-analysis criterion; one resource-bound partial result is excluded and retained in the artifact. Duplicate templates could inflate prevalence, so we fingerprint source token streams and repeat project-level measurements after retaining one representative per exact- and near-clone cluster. The small change in detection rates under both sensitivity analyses reduces, but does not eliminate, concerns about representativeness. Marketplace and industrial applications remain necessary for broader external validity.

Configuration validity. Source/sink definitions, package migration rules, SDK version, call-graph mode, and resource bounds all influence static-taint results. Our tool comparison fixes the HapBench cases, API-17 SDK, and query, reports complete versus incomplete executions, and uses the author-released HapFlow artifact. We intentionally omit the earlier API-20-rule/API-17-loader coverage percentage because it confounded SDK availability, overload metadata, and schema capability. The 716-entry detector rule set is deduplicated and hashed; the release also records 67 explicit category corrections. Separately, the IFDS loader accepts 257 typed privacy-data rules and two explicit framework inputs after removing 655 untyped detector-to-source promotions and resolving a further set of unknown, stale, or externally scoped carriers. An executable SDK audit confirms that each retained privacy-data rule resolves to one API-20 declaration with a matching carrier. These checks eliminate concrete classes of invalid seeds, overload ambiguity, and category pollution, but they still do not prove semantic completeness: the artifact does not establish that all SDK privacy APIs are covered or that every retained category and carrier matches an independent expert oracle. Android analyzers are not executed because they consume Dalvik/Android inputs and framework models; their published numbers would not answer the same ArkTS query.

Asynchronous-analysis validity. HapBench isolates the core IR/dispatch/lifecycle mechanisms, while ArkAsyncBench supplies a balanced source-first oracle for T1/T4/T5 carrier recovery. Its provenance decomposition shows dual core/supplement support for T1 and T5 and a nine-case supplement-exclusive T4 increment. The 26-project diagnostic confirms eight corresponding real-code paths spanning all three constructs. The analysis contract covers these typed callback, Promise-then, and await patterns; arbitrary Promise combinators and framework-defined callback lifetimes remain outside this experiment’s oracle.

6.3 Limitations

ARKPRISM is static and source-based. Its contract excludes runtime-loaded code, native C/C++ bridges, server-side policy, and user-consent state; cross-language analysis such as HarmoBridge [18] is complementary. Receiver-origin recovery is bounded, and the asynchronous supplement targets the ArkTS continuation patterns formalized in Section 3.7 rather than arbitrary event-loop behavior. The reported subgraphs are therefore may-analysis evidence: API presence and a source-to-sink path support code review but do not by themselves establish a legal or policy violation.

7 Related Work

7.1 Source Identity and Privacy API Recognition

Accurate source/sink specifications are critical for mobile privacy analysis. Android studies have mined permission mappings and source/sink lists via Stowaway, PScout, SuSi, Axlplorer, and API-documentation analyses [6, 7, 21, 43, 45], and audited privacy behavior by connecting code, policies, and permissions [3, 55, 62, 63]. These works show that source/sink definitions encode platform semantics and strongly affect reported results.

ARKPRISM applies this principle to OpenHarmony, where package migration from @ohos.* to consolidated @kit.* exports compounds the challenge. Its matching function separates API identity from exact package spelling while retaining namespace constraints. HapFlow’s released Json2ArkMethodSignature loader resolves namespace/method entries to SDK declarations, but its rule schema does not encode executable fields or detector-side factory-origin witnesses [15]. This is a limitation of the released configuration path, not of method signatures in general: a signature analysis with precise receiver types can represent manager methods. ArkAnalyzer provides ArkTS code representation and call-graph construction [14] but does not define privacy API identity resolution. ARKPRISM adds that evidence-carrying layer.

7.2 Asynchronous Semantics and Lifecycle Modeling

Mobile analysis must model lifecycle, callbacks, and implicit edges precisely. Android ICC modeling includes ComDroid, CHEX, Epicc, IC3, IccTA, DidFail, and RAICC [19, 29, 31, 36, 38, 39, 48]; EdgeMiner recovers implicit framework callbacks [13]. Hybrid-app analyses bridge Java, JavaScript, and WebView semantics [11, 30, 57, 58]. These studies show that missing implicit edges dominate mobile analysis errors.

OpenHarmony has analogous but different callback problems: ArkUI declarative builders, class-field callbacks, FunctionType/ClosureType values, and Promise handlers create edges absent from ordinary calls. WEMINT [54] demonstrates explicit callback modeling for asynchronous mini-program APIs via taint rules; ARKPRISM differs by resolving callbacks from ArkAnalyzer’s FunctionType/ClosureType IR and by handling ArkUI builder-option and receiver-factory semantics. HapFlow handles closure semantics [15]; ARKPRISM adds a bounded, separately provenance-labeled post-IFDS recovery for unresolved callback, Promise .then(), and await paths, plus report-CG

edges for field and builder callbacks. The lifecycle state machine (Section 3.6) captures ability/component phase transitions that the flat DummyMain cannot express.

7.3 Evidence Representation and Large-scale Auditing

Android taint analysis has a rich history. TaintDroid [20] motivated smartphone privacy monitoring; FlowDroid [5] introduced context-/flow-/field-/object-sensitive lifecycle-aware analysis; Amandroid, DroidSafe, IccTA, LeakMiner, and AndroidLeaks explored inter-procedural data-flow, vetting, and market-side detection [25, 26, 31, 52, 61]; HornDroid and COVERT added formal and compositional perspectives [8, 12]. More recently, PacDroid unifies pointer analysis, inter-procedural propagation, ICC, and lifecycle handling and evaluates the resulting soundness–precision–performance tradeoff against several Android frameworks [16]. These systems are important methodological references, but they consume Android bytecode and Android framework models; running them on ArkTS would change both the input language and the analysis query. We therefore do not present their published numbers as an empirical baseline for ARKPRISM.

ARKPRISM uses the author-released HapFlow artifact as the directly comparable OpenHarmony taint baseline and adds recognition and subgraph evidence around it. This choice is task based rather than reputation based: HapFlow accepts ArkTS projects and answers the same source-to-sink reachability query. ArkAnalyzer and its context-sensitive Pointer Analysis Kit (APAK) provide the source IR, call graphs, and points-to information on which a taint client can be built [14, 60], but do not themselves emit privacy source-to-sink findings. HarmonyFlow instead analyzes compiled Ark Panda IR and evaluates pointer/call-edge recovery [51]; its input representation and output query differ from ours. We therefore discuss these systems as strong front-end alternatives and design influences, not as fabricated end-to-end baselines. TaintMini’s Universal Data Flow Graph [33] is a core analysis intermediate representation that drives cross-page data-flow resolution in mini-programs; ARKPRISM’s evidence subgraph differs in that it is a post-hoc audit artifact assembled from detection, call-chain, sink, and taint results with provenance tracking, rather than a structure that drives the analysis algorithm itself.

Malware and behavior research uses permissions, API calls, graphs, and models to classify malicious apps [1, 4, 22, 24, 40, 44, 49, 53, 56, 59, 64–66]; AndroZoo enables ecosystem-scale studies [2]. ARKPRISM follows this empirical principle: the information-flow subgraph is more structured than a flat feature vector because auditors need source snippets, call contexts, sink classes, and metadata.

ARKPRISM builds on IFDS/IDE foundations [46, 47] and reusable infrastructures (Soot, WALA, Heros [9, 23, 50]). ArkAnalyzer plays this role for ArkTS [14]. APAK reports that ArkTS closure and framework modeling materially changes downstream false positives relative to CHA/RTA [60]; this motivates our PTA ablation and our separate reporting of virtual-dispatch errors. Other OpenHarmony efforts explore testing [17, 35], repair [34], and compiled-IR analysis [51], showing that OpenHarmony requires ecosystem-specific analysis [27, 32, 41]. ARKPRISM follows the same layered principle: reuse the framework, then add domain-specific recognition and provenance-annotated subgraph mapping.

7.4 Evaluation Methodology for Mobile Analyses

Comparative studies show that mobile-analysis results are easily distorted by different source/sink lists, different queries, incomplete benchmark subsets, and undocumented ground truth. ReproDroid evaluates six Android taint analyzers under a common query and execution framework [42]. TaintBench goes further by storing expected and unexpected real-world flows in a machine-readable format and validating labels through multi-expert review [37]. Mutation-based evaluation complements fixed benchmarks by systematically exposing unsound analysis choices [10]. These

studies shape our protocol: we use the complete author-released HapBench suite, publish a case-level oracle, keep the source and sink query fixed, count incomplete executions separately, report both positive- and negative-case metrics, and retain every misclassified case. The high-output real-project audit is reported as a precision-oriented audit rather than an independent recall benchmark because projects were selected using ARKPRISM’s output.

8 Conclusion

We presented ARKPRISM, an evidence-preserving static analysis for resolving privacy-API identities and carrying typed source values across ArkTS IR and selected asynchronous continuations. Its package migration, receiver-origin, property, and IR witnesses recover identities that cannot be decided from a bare member token, while provenance keeps core IFDS paths distinct from bounded asynchronous supplementation. On a source-first real-project benchmark, ARKPRISM resolves all 90 gold occurrences at exact source locations and rejects every candidate collision, including 44 zero projects; a 64-case identity benchmark attributes this result to the composition of import, receiver, factory, property, and alias evidence. On HapBench, ARKPRISM combines 100% precision and specificity with 97.09% F1, and the asynchronous benchmark isolates nine Promise-then paths recovered by the supplement. The single-build 1,014-project study demonstrates that the same analysis and evidence contract scales to a heterogeneous ArkTS corpus. Together, these results establish source identity as a first-class, auditable stage of ArkTS privacy analysis rather than an implicit configuration side effect.

Data Availability

The artifact contains the ARKPRISM source and build scripts, fixed source/sink and package-migration rules, the source-first 60-project occurrence benchmark, the 120-project high-output stress audit, the 64-case identity benchmark, the 67-case HapBench and 48-case ArkAsyncBench oracles and raw outputs, per-project reports for the 1,014 completed corpus analyses, and scripts that regenerate the evaluation tables and figures. Dataset redistribution follows the licenses of the constituent projects; where redistribution is restricted, the artifact provides project identifiers, hashes, and derived reports.

A Reproduction Details

A.1 HapBench Case-level Outcomes

Table 8 lists every error produced by either executable tool under the fixed 67-case oracle. Cases absent from the table are classified correctly by both tools. We retain the source-annotated oracle even where a more restrictive event or object-lifetime semantics would suggest a different label.

Table 8. Complete error inventory for the main HapBench comparison.

Tool	Outcome	Case	Observed mechanism
HapFlow	FN	AnonymousClass2	Anonymous instance initializer omitted
HapFlow	FN	Exceptions4	Exceptional successor/handler not recovered
HapFlow	FP	ArrayIndexNoLeak	Distinct constant indices collapse
HapFlow	FP	VirtualDispatch2	Infeasible receiver target retained
HapFlow	FP	VirtualDispatch3	Infeasible receiver target retained
HapFlow	FP	UnreachableFlow	Lifecycle cycle creates reachability
ARKPRISM	FN	ActivityLifecycle4	Parent source method is bypassed by child override
ARKPRISM	FN	BackupExtensionAbility	Oracle assumes state across extension instances
ARKPRISM	FN	Button1	Oracle assumes click-written static state reaches later creation

The two tools agree on 58 outcomes. Of the nine discordant cases, ARKPRISM alone is correct on six and HapFlow alone on three. The exact two-sided McNemar test is therefore based on the discordant pair (6, 3) and yields $p = 0.508$.

A.2 Lifecycle-Oracle Sensitivity

The primary evaluation preserves all published/source-annotated HapBench labels. We additionally define a strict executed-order contract for the three disputed lifecycle cases: an override does not execute its parent body without an explicit super call; an instance field does not cross framework-created instances without a same-object witness; and a UI callback occurring after initial ability creation does not feed an earlier onCreate without an explicit later-creation edge. This contract relabels ActivityLifecycle4, BackupExtensionAbility, and Button1 as negative and changes no other case.

Table 9. Sensitivity to strict lifecycle execution order.

Tool	TP	TN	FP	FN	Precision	Recall	Specificity	F1	MCC
HapFlow [15]	48	10	7	2	87.27%	96.00%	58.82%	91.43%	0.622
ARKPRISM	50	17	0	0	100.00%	100.00%	100.00%	100.00%	1.000

The tools then disagree on nine cases, all classified correctly only by ARKPRISM; the exact two-sided McNemar test gives $p = 0.00390625$. This is a post-hoc sensitivity analysis rather than a replacement oracle. It shows that the primary nonsignificant result is conservative with respect to these three lifecycle assumptions, while also making the semantics responsible for the difference explicit. The artifact derives both tables from the same case predictions through `analyze_hapbench_oracle_sensitivity.js`.

A.3 Uncertainty and Category Robustness

Table 10 reports Wilson intervals for the main configurations. The wide specificity intervals are a direct consequence of having only 14 negative cases; balanced accuracy and MCC are included to make this class imbalance visible.

Table 10. HapBench uncertainty and category robustness.

Tool	Precision 95% CI	Recall 95% CI	Specificity 95% CI	Macro F1	Worst-category F1
HapFlow [15]	[82.7, 97.1]	[87.2, 99.0]	[45.4, 88.3]	95.73%	88.89%
ARKPRISM	[92.9, 100.0]	[84.6, 98.1]	[78.5, 100.0]	97.96%	85.71%
–IR recovery	[90.6, 100.0]	[56.5, 80.5]	[78.5, 100.0]	87.07%	66.67%
–receiver refinement	[89.7, 99.7]	[84.6, 98.1]	[68.5, 98.7]	97.57%	85.71%
Unbounded lifecycle	[90.1, 99.7]	[90.1, 99.7]	[68.5, 98.7]	98.81%	91.67%

Table 11. Per-category F1 on all HapBench cases.

Category	Cases	HapFlow	ARKPRISM
Aliasing	6	100.00%	100.00%
Anonymous constructs	10	93.33%	100.00%
Array-like structures	5	88.89%	100.00%
Field/object sensitivity	6	100.00%	100.00%
General language features	22	91.89%	100.00%
Lifecycle modeling	13	96.00%	85.71%
OpenHarmony-specific APIs	5	100.00%	100.00%

A.4 Source-first Benchmark Protocol

ArkSourceFirst60 is selected before the evaluated detector run. Projects are partitioned by source-tree size and by whether their imports are compatible with the frozen sensitive-API rules; deterministic hash ordering selects ten projects from each of the six strata. The manifest stores the selection seed, source-tree fingerprint, rule hash, and package-migration hash. Candidate discovery parses executable member accesses and import bindings, but it neither reads a ARKPRISM report nor assigns a gold label.

Manual review accepts an occurrence only when its executable source context establishes the configured package/namespace/member and access kind through an import, SDK receiver type, constructor, manager/factory definition, or executable property read. It rejects comments, strings, declarations, same-name application methods, and unrelated library/controller receivers. The delivered gold file contains 90 accepted occurrences; a companion decision file preserves all 101 accept/reject judgments, including the 11 high-confusion negatives. An integrity test rehashes every source file, validates every exact line and canonical rule key, and rejects duplicate occurrence identities. Evaluation joins detector and gold records on project, normalized file, exact line, canonical API, and access kind.

A.5 High-output Stress-audit Decision Rule

The 120-project audit labels an item at project–namespace–member level. A report is accepted only when the recorded source context contains an executable access matching the configured API and one of the following identity witnesses:

- an imported namespace or package alias invokes the configured member;
- a manager/helper receiver is connected to its configured factory by a visible definition or field assignment;
- an SDK receiver type identifies the configured namespace; or
- an executable field/property access is declared as a property source in the fixed rule set.

Comments, string literals, declarations without access, unrelated application methods with the same final token, and ambiguous method-only matches across multiple package namespaces are rejected. For every accepted key, the artifact stores project identifier, normalized package/namespace/member, access kind, source file and line, and a bounded source snippet. Precision is computed over the 666 project–API keys.

The enriched construction file contained 1,533 report-derived rows, including repeated rows that pointed to the same source evidence. The delivered `top120_source_audit_annotations.json` canonicalizes them to the 666 audited project–API keys, removes detector-internal fields, and does not contain occurrence-precision, false-negative, or recall counters. Its declared scope is `output_selected_precision_audit`; both `occurrencePrecisionDefined` and `recallDefined` are false. This prevents the source-evidence artifact from being mistaken for an exhaustive occurrence-level or source-first oracle.

The companion integrity audit independently resolves each record against the delivered source tree and final detector rules. It checks project and file existence, positive line bounds, member text on the recorded line, nonempty snippets, and package/namespace/member compatibility under the same migration and normalization contract as the detector. All 666 records pass these structural checks. This check establishes artifact referential integrity; it does not replace the source-review decision or convert the audit into a recall benchmark.

A.6 Artifact-to-Paper Traceability

The artifact provides ArkSourceFirst60’s selection manifest, review decisions, occurrence gold set, and integrity evaluator; the 64-case ArkIdentityBench progression oracle; machine-readable oracles for all 67 HapBench and 48 ArkAsyncBench cases; and raw output for every evaluated configuration. The evaluators derive confusion matrices, construct/category metrics, paired outcomes, and runtimes. Figure 3 is generated from evaluator JSON rather than transcribed from the paper. Corpus tables and figures consume one process-isolated run of the final build; change-impact overlays are rejected by the release summarizer. The run manifest and batch summary record the project inventory, analyzer revision and build hash, rule and SDK fingerprints, command line, resource bounds, completion state, and the explicitly excluded partial project.

The corpus audit checks three invariants before aggregation: each configured-query path has resolvable source and sink endpoints, each path carries an `ifds`, `async_supplement`, or both provenance tag, and every detector-to-path link references valid records with compatible source evidence. Detector-local sink records and configured-query paths remain separate when this conservative join cannot be established. Figure companion JSON files retain the input manifest and computed values so that figures, tables, and prose cannot silently use different runs.

References

- [1] Yousra Aafer, Wenliang Du, and Heng Yin. 2013. DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android. In *Proceedings of the International Conference on Security and Privacy in Communication Systems*. 86–103.
- [2] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories (ICMSR)*. ACM, New York, NY, USA, 468–471.

- [3] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck. In *29th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online, 985–1002.
- [4] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, Konrad Rieck, and CERT Siemens. 2014. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the Network and Distributed System Security Symposium*.
- [5] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Oteau, and Patrick McDaniel. 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, New York, NY, USA, 259–269.
- [6] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proceedings of the ACM Conference on Computer and Communications Security*. 217–228.
- [7] Michael Backes, Sven Bugiel, Erik Derr, Patrick McDaniel, Damien Oteau, and Sebastian Weisgerber. 2016. On Demystifying the Android Application Framework: Re-Visiting Android Permission Specification Analysis. In *Proceedings of the USENIX Security Symposium*. 1101–1118.
- [8] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. COVERT: Compositional Analysis of Android Inter-App Permission Leakage. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*. 866–877.
- [9] Eric Bodden. 2012. Inter-procedural Data-flow Analysis with IFDS/IDE and Soot. In *Proceedings of the ACM SIGPLAN International Workshop on State of the Art in Java Program Analysis*. 3–8.
- [10] Richard Bonett, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. 2018. Discovering Flaws in Security-Focused Static Analysis Tools for Android Using Systematic Mutation. In *Proceedings of the 27th USENIX Security Symposium*. 1263–1280. <https://www.usenix.org/conference/usenixsecurity18/presentation/bonett>
- [11] Achim D. Brucker and Michael Herzberg. 2018. BabelView: Evaluating the Impact of Code Injection Attacks in Mobile Webviews. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 25–46.
- [12] Stefano Calzavara, Ilya Grishchenko, and Matteo Maffei. 2016. HornDroid: Practical and Sound Static Analysis of Android Applications by SMT Solving. In *Proceedings of the IEEE European Symposium on Security and Privacy*. 47–62.
- [13] Yinzhao Cao, Yanick Fratantonio, Antonio Bianchi, Antonio Chung, Yanick Huang, David Lie, Giovanni Vigna, and Christopher Kruegel. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *Proceedings of the Network and Distributed System Security Symposium*.
- [14] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE SEIP)*. ACM.
- [15] Haonan Chen, Mingyi Zhou, Yanjie Zhao, and Li Li. 2026. HapFlow: The Taint Analysis Framework for OpenHarmony Apps. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3793863
- [16] Menglong Chen, Tian Tan, Minxue Pan, and Yue Li. 2025. PacDroid: A Pointer-Analysis-Centric Framework for Security Vulnerabilities in Android Apps. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. 2803–2815. doi:10.1109/ICSE55347.2025.00208
- [17] Yige Chen, Yujia Fan, Siyi Wang, Yi Tao, and Yepang Liu. 2025. HmTest: Automated Testing of HarmonyOS Apps via Model-Driven Navigation and Reinforcement Learning. *Journal of Computer Science and Technology* 40, 4 (2025), 1006–1021. doi:10.1007/s11390-025-5142-4
- [18] Zhen Chen, Yi Li, Teng Wang, et al. 2025. HarmoBridge: Static Analysis of Cross-Language Data Flows in HarmonyOS Applications. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [19] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. 2011. Analyzing Inter-Application Communication in Android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services*. 239–252.
- [20] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2014. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. *ACM Trans. Comput. Syst.* 32, 2 (2014), 5:1–5:29.
- [21] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android Permissions Demystified. In *Proceedings of the ACM Conference on Computer and Communications Security*. 627–638.
- [22] Yu Feng, Saswat Anand, Isil Dillig, and Alex Aiken. 2014. Apposcopy: Semantics-Based Detection of Android Malware through Static Analysis. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 576–587.
- [23] Stephen J. Fink, Julian Dolby, and Eran Yahav. 2008. Effective Typestate Verification in the Presence of Aliasing. *ACM Transactions on Software Engineering and Methodology* 17, 2, Article 9 (2008).

- [24] Yanick Fratantonio, Antonio Bianchi, William Robertson, Dhillung Kirat, Christopher Kruegel, and Giovanni Vigna. 2016. TriggerScope: Towards Detecting Logic Bombs in Android Applications. In *Proceedings of the IEEE Symposium on Security and Privacy*. 377–396.
- [25] Clint Giber, Jonathan Crussell, Jeremy Erickson, and Hao Chen. 2012. AndroidLeaks: Automatically Detecting Potential Privacy Leaks in Android Applications on a Large Scale. In *Proceedings of the 5th International Conference on Trust and Trustworthy Computing*. 291–307.
- [26] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *Proceedings of the Network and Distributed System Security Symposium*.
- [27] Huawei. 2021. HarmonyOS 2 Security Technical White Paper. <https://consumer.huawei.com/>
- [28] Huawei Developer. 2026. ArkTS: Dedicated Programming Language for the HarmonyOS Ecosystem. <https://developer.huawei.com/consumer/en/arkts/>
- [29] William Klieber, Lori Flynn, Amar Bhosale, Limin Jia, and Lujo Bauer. 2014. Android Taint Flow Analysis for App Sets. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 1–6.
- [30] Sungcho Lee, Julian Dolby, and Suhyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 250–261. doi:10.1145/2970276.2970368
- [31] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Outeau, and Patrick McDaniel. 2015. IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In *Proceedings of the 37th International Conference on Software Engineering*. 280–291.
- [32] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé F. Bissyandé, Jacques Klein, John Grundy, Tao Xie, Haibo Chen, and Huaimin Wang. 2025. Software Engineering for OpenHarmony: A Research Roadmap. *Comput. Surveys* 58, 2, Article 34 (2025), 34 pages. doi:10.1145/3720538
- [33] Yi Li, Yanyan Chen, Teng Wang, et al. 2023. TaintMini: Detecting Privacy Data Leaks in Mini-Programs via Universal Data Flow Graph. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*.
- [34] Zhihao Lin, Mingyi Zhou, Wei Ma, Chi Chen, Yun Yang, Jun Wang, Chunming Hu, and Li Li. 2025. HapRepair: Learn to Repair OpenHarmony Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Industry Track*. 319–330. doi:10.1145/3696630.3728556
- [35] Farong Liu, Mingyi Zhou, Yakun Zhang, Ting Su, Bo Sun, Jacques Klein, Xiang Gao, and Li Li. 2025. HapTest: The Dynamic Analysis Framework for OpenHarmony. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, Companion Proceedings*. 1–10. doi:10.1145/3696630.3728565
- [36] Long Lu, Zhichun Li, Zhenyu Wu, Wenke Lee, and Guofei Jiang. 2012. CHEX: Statically Vetting Android Apps for Component Hijacking Vulnerabilities. In *Proceedings of the ACM Conference on Computer and Communications Security*. 229–240.
- [37] Linghui Luo, Felix Pauck, Goran Piskachev, Manuel Benz, Ivan Pashchenko, Martin Mory, Eric Bodden, Ben Hermann, and Fabio Massacci. 2022. TaintBench: Automatic Real-World Malware Benchmarking of Android Taint Analyses. *Empirical Software Engineering* 27, 1 (2022), 16. doi:10.1007/s10664-021-10013-5
- [38] Damien Outeau, Somesh Jha, and Patrick McDaniel. 2013. Effective Inter-Component Communication Mapping in Android with Epicc: An Essential Step Towards Holistic Security Analysis. In *Proceedings of the 22nd USENIX Security Symposium*. 543–558.
- [39] Damien Outeau, Daniel Luchaup, Matthew Dering, Somesh Jha, and Patrick McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *Proceedings of the 37th International Conference on Software Engineering*. 77–88.
- [40] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2019. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models (Extended Version). *ACM Transactions on Privacy and Security (TOPS)* 22, 2, Article 10 (2019), 10:1–10:29 pages.
- [41] OpenAtom Foundation. 2026. OpenHarmony: A Comprehensive Open Source Project for the All-scenario, Fully-connected, Intelligent Era. <https://www.openharmony.cn/>
- [42] Felix Pauck, Eric Bodden, and Heike Wehrheim. 2018. Do Android Taint Analysis Tools Keep Their Promises?. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 331–341. doi:10.1145/3236024.3236029
- [43] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks. In *Proceedings of the Network and Distributed System Security Symposium*.
- [44] Siegfried Rasthofer, Steven Arzt, Enrico Lovat, and Eric Bodden. 2016. HARVESTER: Automatic Generation of Entry Points for Dynamic Analysis of Android Applications. In *Proceedings of the Network and Distributed System Security Symposium*.

- [45] Talia Ravitch, Eric Creswick, Aaron Tomb, Adam Foltzer, Trevor Elliott, and Ledah Casburn. 2014. Multi-App Security Analysis with Fuse: Statically Detecting Android App Collusion. In *Proceedings of the ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 4–9.
- [46] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 49–61.
- [47] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theoretical Computer Science* 167, 1–2 (1996), 131–170.
- [48] Jordan Samhi, Alexandre Bartel, Tegawendé F. Bissyandé, and Jacques Klein. 2021. RAICC: Revealing Atypical Inter-Component Communication in Android Apps. *Empirical Software Engineering* 26, 6 (2021).
- [49] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallaro. 2015. CopperDroid: Automatic Reconstruction of Android Malware Behaviors. In *Proceedings of the Network and Distributed System Security Symposium*.
- [50] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 2010. Soot: A Java Bytecode Optimization Framework. In *CASCON First Decade High Impact Papers*. IBM Corp., 214–224.
- [51] Yue Wang, Sen Chen, Jiaming Li, Wenjing Dang, Renchao Chai, Ziyue Shi, and Li Li. 2025. HarmonyFlow: A Static Analysis Framework for HarmonyOS Applications Based on Ark Panda IR. *Journal of Software* (2025), 1–19. doi:10.13328/j.cnki.jos.007477
- [52] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Trans. Priv. Secur.* 21, 3 (2018), 14:1–14:32.
- [53] Michelle Y. Wong and David Lie. 2016. IntelliDroid: A Targeted Input Generator for the Dynamic Analysis of Android Malware. In *Proceedings of the Network and Distributed System Security Symposium*.
- [54] Haoyu Wu, Lingxiang Yang, Mengfei Wang, et al. 2023. WEMINT: Detecting Sensitive Data Flows in Mini-Programs via Explicit Asynchronous Callback Modeling. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [55] Meng Xia, Lu Gong, Yuan Lyu, Zhengwei Qi, and Xue Liu. 2015. Effective Real-time Android Application Auditing. In *Proceedings of the IEEE Symposium on Security and Privacy*. 899–914.
- [56] Lok Kwong Yan and Heng Yin. 2012. DroidScope: seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis. In *Proceedings of the 21st USENIX Conference on Security Symposium (USENIX Security)* (Bellevue, WA). USENIX Association, USA, 29.
- [57] Guangliang Yang, Abner Mendoza, and Guofei Gu. 2018. Automated Generation of Event-Oriented Exploits in Android Hybrid Apps. In *Proceedings of the Network and Distributed System Security Symposium*.
- [58] Guangliang Yang, Abner Mendoza, Jialong Zhang, and Guofei Gu. 2017. BridgeScope: Precisely and Scalably Vetting JavaScript Bridge Security Issues in Android Hybrid Apps. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 166–185.
- [59] Wei Yang, Xusheng Xiao, Benjamin Andow, Sihan Li, Tao Xie, and William Enck. 2015. AppContext: Differentiating Malicious and Benign Mobile App Behaviors Using Context. In *Proceedings of the International Conference on Software Engineering*. 303–313.
- [60] Yizhuo Yang, Lingyun Xu, Mingyi Zhou, and Li Li. 2026. Context-Sensitive Pointer Analysis for ArkTS. arXiv preprint. <https://arxiv.org/abs/2602.00457>
- [61] Zheming Yang and Min Yang. 2012. LeakMiner: Detect Information Leakage on Android with Static Taint Analysis. In *Proceedings of the 3rd World Congress on Software Engineering*. 101–104.
- [62] Zheming Yang, Min Yang, Yuan Zhang, Guofei Gu, Peng Ning, and X. Sean Wang. 2013. AppIntent: Analyzing Sensitive Data Transmission in Android for Privacy Leakage Detection. In *Proceedings of the ACM Conference on Computer and Communications Security*. 1043–1054.
- [63] Le Yu, Xiapu Luo, Jiachi Chen, Hao Zhou, Tao Zhang, Henry Chang, and Hareton K. N. Leung. 2021. PPChecker: Towards Accessing the Trustworthiness of Android Apps' Privacy Policies. *IEEE Transactions on Software Engineering (TSE)* 47, 2 (2021), 221–242.
- [64] Mu Zhang, Yue Duan, Heng Yin, and Zhiruo Zhao. 2014. Semantics-Aware Android Malware Classification Using Weighted Contextual API Dependency Graphs. In *Proceedings of the ACM Conference on Computer and Communications Security*. 1105–1116.
- [65] Wu Zhou, Yajin Zhou, Xuxian Jiang, and Peng Ning. 2012. Detecting Repackaged Smartphone Applications in Third-party Android Marketplaces. In *Proceedings of the ACM Conference on Data and Application Security and Privacy*. 317–326.
- [66] Yajin Zhou, Zhi Wang, Wu Zhou, and Xuxian Jiang. 2012. DroidRanger: Systematic Characterization and Detection of Malicious Applications in Official and Alternative Android Markets. In *Proceedings of the Network and Distributed System Security Symposium*.